

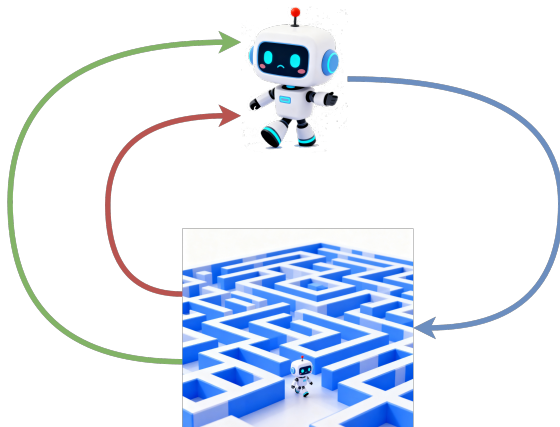
1. 强化学习概述

1.1 什么是强化学习

强化学习（Reinforcement Learning, RL）是机器学习的一个重要分支，它关注智能体（Agent）如何在复杂、不确定的环境中通过与环境的交互来学习最优的行为策略。与监督学习和无监督学习不同，强化学习没有明确的标准答案或标签数据，而是通过环境反馈的奖励信号来指导学习过程。

(1) 强化学习的思想

强化学习的基本思想源于心理学中的条件反射理论。在经典的条件反射实验中，实验对象通过接受正向或负向的刺激，逐渐学会将特定的行为与相应的结果关联起来。类似地，强化学习中的智能体通过执行动作并观察环境给出的奖励或惩罚，逐步调整自己的行为策略，以获得最大的累积奖励。



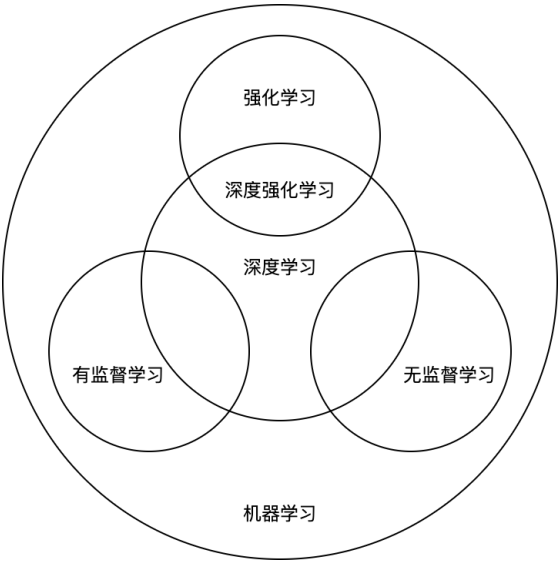
这种学习机制体现了"试错学习"（Trial-and-Error Learning）的特点：智能体在不断尝试不同行动的过程中，通过分析行动的后果来改进自己的决策。当某个行动导致正向奖励时，智能体会增加选择该行动的概率；当某个行动导致负向奖励时，智能体会降低选择该行动的概率。这种基于奖励反馈的学习方式使得智能体能够在复杂环境中自主地发现有效的行为模式。

(2) 强化学习在机器学习中的位置

在机器学习的框架中，强化学习与监督学习、无监督学习构成了三种主要的学习范式。监督学习通过大量的输入-输出样本来训练模型，学习从输入到输出的映射关系。这种学习方式需要明确的标签信息，适用于分类和回归问题。无监督学习则在没有标签信息的情况下，通过发现数据中的隐含结构和模式来进行学习，主要应用于聚类、降维和密度估计等任务。强化学习介于两者之间，虽然没有明确的标签，但有来自环境的奖励信号作为学习的指导。这种学习方式特别适合于需要序列决策的问题，如游戏策略、机器人控制、自动驾驶等领域。

深度学习方法与强化学习的结合催生了**深度强化学习**（Deep Reinforcement Learning），它利用深度神经网络强大的特征表示能力来解决高维感知输入和复杂环境下的决策问题。这一方向推动了人工智能的突破性进展，例如 AlphaGo、无人驾驶等。此外，强化学习在大模型的训练和优化中同样发挥着重要作用。例如，**基于人类反馈的强化学习**（RLHF, Reinforcement Learning with Human Feedback）已经成为大语言模型（LLM）对齐的重要方法。通过结合人类偏好与奖励建模，强化学习能够引导大模型生成更加符合人类价值观和使用需求的输出。类似的思路还被扩展到在线优化、模型微调和多模态任务中，为大模型的发展提供了新的训练范

式。由此可见，强化学习不仅在序列决策问题中不可或缺，也在推动下一代人工智能系统的落地与进步中扮演着核心角色。



1.2 强化学习环境

强化学习的实验和应用需要合适的环境平台来支持智能体的训练和测试。这些环境为算法研究者提供了标准化的测试平台，使得不同算法之间的比较成为可能。本部分介绍三种常用的强化学习王朝。Grid World是一种简单的环境，适合于概念验证和学习，Gym/Gymnasium环境适合于算法比较和性能评估，而MuJoCo环境则适合于实际应用的开发和测试。除此之外，常用的强化学习环境还有Atari（游戏环境）、PyBullet（物理仿真和机器人控制）等。

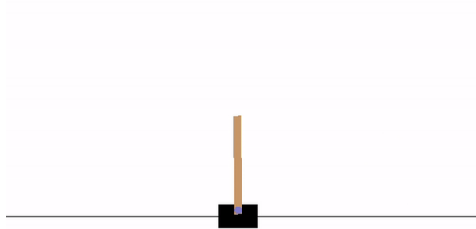
由于Gym / Gymnasium 风格的接口已经成为强化学习环境的事实标准，因此常用环境几乎都能通过Gym/Gymnasium接口使用。

(1) Grid World环境

Grid World是强化学习中最经典的环境之一，它将复杂的决策问题简化为在网格世界中的导航任务。在这个环境中，智能体位于一个二维网格中，需要通过选择移动方向（上、下、左、右）来到达目标位置，同时避开障碍物或陷阱。

Grid World环境的优势在于其简单性和可视化特性。研究者可以直观地观察智能体的学习过程，理解算法的工作原理。这个环境通常用于验证新算法的正确性，或者作为教学工具来帮助学习者理解强化学习的基本概念。

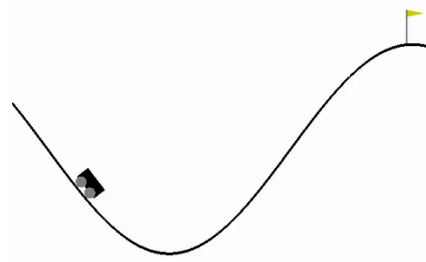
尽管Grid World看似简单，但它包含了强化学习的所有关键要素：状态空间（网格位置）、动作空间（移动方向）、奖励函数（到达目标的奖励和撞墙的惩罚）以及状态转移（移动的结果）。通过调整网格大小、障碍物分布和奖励设置，可以创建不同复杂度的学习任务。



CartPole



Pendulum



MountainCarContinuous

- 02-gymnasium.py

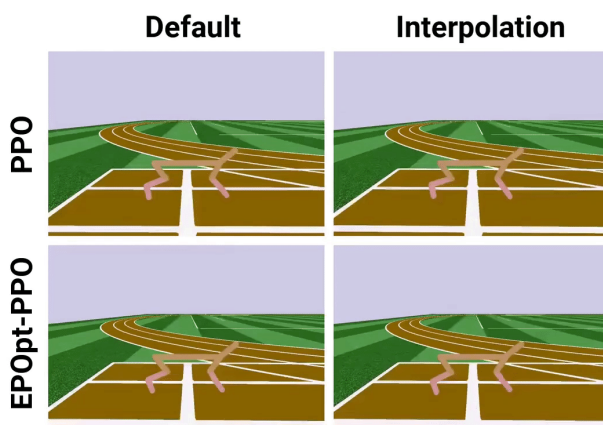
(3) MuJoCo物理引擎

MuJoCo (Multi-Joint Dynamics with Contact) 是一个高性能的物理引擎，专门用于机器人学和生物力学研究。它能够精确模拟多关节系统的动力学特性，支持复杂的接触和碰撞检测。在强化学习领域，MuJoCo 被广泛用于连续控制任务中的仿真与算法测试。

自 2021 年起，MuJoCo 已由 **DeepMind 收购并开源**，目前在 GitHub 上持续维护，免费向学术界和工业界开放。这一转变不仅降低了使用门槛，也促进了其与强化学习平台的深度结合。尤其是随着 **Gymnasium 的推出**，MuJoCo 已经通过官方接口与 Gymnasium 环境整合，研究者可以像使用其他标准环境一样，直接调用 MuJoCo 驱动的仿真任务。

MuJoCo 支持多种复杂的机器人模型，包括人形机器人、四足机器人和机械臂等。其高保真的物理模拟能够帮助研究者在仿真中安全地测试先进的学习算法，从而在一定程度上提升算法向真实机器人系统迁移的可能性。

另一个重要特性是其高效的计算性能。MuJoCo 针对复杂系统的动力学计算进行了优化，能够快速生成大规模仿真数据，这对于样本需求量极大的强化学习算法至关重要。此外，MuJoCo 还提供了丰富的传感器模型，包括位置、速度、力等，使得仿真环境更加贴近真实情况，进一步提升了其在机器人研究中的价值。



- 03-mojoco.py

2. 强化学习基本概念

2.1 强化学习的一般流程

(1) 强化学习的要素

一个强化学习系统，无论其复杂程度如何，都包含以下几个基本构成要素：

- **智能体 (Agent)**：学习者和决策者。它是算法的核心，负责观察环境、做出决策（选择动作），并根据反馈进行学习。
- **环境 (Environment)**：智能体之外的一切。它定义了世界的规则，接收智能体的动作，并给出新的状态和奖励。在不同的场景下，智能体对环境有不同的掌握和感知程度。
 - **完全可观测环境**，是指智能体能够完全感知环境的所有相关状态信息。在这种情况下，智能体具有环境的完整模型，包括状态转移概率和奖励函数。完全可观测环境通常可以使用动态规划方法来求解最优策略。
 - **部分可观测环境**，是指智能体只能观察到环境状态的一部分信息，或者对环境的动态特性了解有限。这种情况更接近现实世界的问题，智能体需要在不完全信息的条件下做出决策。大多数强化学习算法都是为部分可观测环境设计的。
- **状态 (State)**：对环境在某一时刻的描述。它是智能体做出决策所依据的信息。例如，在棋类游戏中，当前棋盘的布局就是一个状态。所有状态的集合构成“状态空间”，**状态空间可以是离散的也可以是连续的**。
- **动作 (Action)**：智能体可以执行的操作。所有可能动作的集合构成了“动作空间”。**动作空间可以是连续的，也可以是离散的**。
- **奖励 (Reward)**：环境在智能体执行一个动作后，反馈给智能体的一个标量信号。它是评价动作好坏的直接标准。奖励可以是正向的（鼓励），也可以是负向的（惩罚）。

智能体的最终目标不是最大化眼前的单步奖励，而是最大化从长远来看的**累积奖励**。



此外，**模型**也是强化学习中的重要概念。强化学习模型是智能体对环境的内部表示和理解，它包括两个关键组件：状态转移函数和奖励函数。**状态转移函数** $P(s'|s, a)$ 描述了在状态 s 下执行行动 a 后转移到状态 s' 的概率。**奖励函数** $R(s, a)$ 或 $R(s, a, s')$ 定义了在一定状态下执行特定行动所获得的即时奖励。基于模型信息的可获得性，强化学习方法也可以分为：

- **基于模型的强化学习 (Model-based RL)** 假设智能体知道或能够学习环境的状态转移函数和奖励函数。在这种情况下，智能体可以使用规划算法（如动态规划）来计算最优策略，或者使用学习到的模型来模拟经验以提高样本效率。
- **无模型强化学习 (Model-free RL)** 不依赖于环境模型的精确知识，而是直接从与环境的交互经验中学习最优策略或价值函数。这类方法更加灵活，适用于模型未知或难以建模的复杂环境。

(2) 智能体与环境的交互

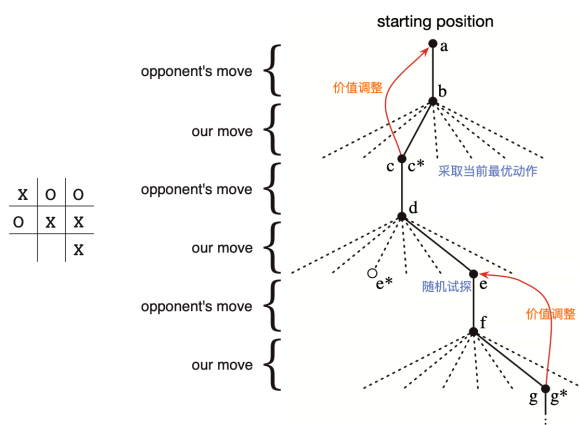
强化学习的“一般流程”指的是智能体与环境之间持续交互的标准化过程。这个过程可以被描述为一个循环：

- **观察状态**：在时间步 t ，智能体观察到环境的当前状态 s_t 。

- **选择动作**：智能体根据其内部的**策略（Policy）**，基于状态 s_t 选择一个动作 a_t 。策略是智能体的“大脑”，定义了特定状态下应该采取何种行动的规则。
- **环境响应**：环境接收到动作 a_t 后，会发生变化，进入下一个状态 s_{t+1} 。同时，环境会给智能体一个即时奖励 r_{t+1} 作为反馈。
- **学习与改进**：智能体利用这次交互产生的经验——即一个四元组 $(s_t, a_t, r_{t+1}, s_{t+1})$ ——来更新和改进自己的策略，以便在未来做出更好的决策。
- **循环往复**：智能体在新状态 s_{t+1} 下，重复上述过程，直到交互结束。

这个从开始到结束的完整交互序列，我们称之为一个**回合（Episode）**。一个回合的结束通常意味着达到了某个终止状态（如游戏胜利或失败）或预设的步数上限。

• 例：井字棋



整个强化学习的训练过程，就是让智能体经历成千上万个回合。在每个回合中，智能体不断地进行“观察-决策-行动-反馈”的循环，并利用积累的经验持续优化其决策策略。这个流程可以用以下伪代码来抽象表示：

循环处理每一个回合（Episode）：

- 初始化环境，获得初始状态 s 。
- 循环处理回合中的每一步（Step）：
 - 智能体根据当前状态 s 和自身策略，选择动作 a 。
 - 在环境中执行动作 a ，获得新状态 s' 和奖励 r 。
 - 智能体利用经验 (s, a, r, s') 进行学习，更新策略。
 - 更新当前状态： $s = s'$ 。
 - 如果 s 是终止状态，则结束本回合。

这个框架是所有强化学习算法的共同基础。不同算法的区别主要在于智能体如何定义其**策略**（步骤 a）以及如何利用经验来**学习**（步骤 c）。例如，有些算法学习的是一个价值函数，用来评估状态或动作的好坏；而另一些算法则直接学习策略本身。

- 例：04-rl_basic_process.py

2.2 序列决策与马尔可夫过程

(1) 序列决策问题

强化学习本质上是一个序列决策问题。智能体在每个时间步都需要根据当前观察到的环境状态来选择一个行动，这个行动会影响环境的下一个状态，进而影响智能体未来可能获得的奖励。因此，智能体需要考虑的不仅仅是当前行动的即时收益，还需要考虑该行动对未来长期收益的影响。

具体而言，智能体与环境的交互过程可以描述为一个时间序列：

$$s_0, a_0, r_1, s_1, a_1, r_2, s_2, a_2, r_3, \dots$$

其中 s_t 表示时间步 t 的环境状态， a_t 表示智能体在状态 s_t 下选择的行动， r_{t+1} 表示执行行动 a_t 后环境给出的奖励。这个序列构成了一个轨迹（trajectory），记作 τ 。

序列决策的目标，就是智能体根据多个回合交互生成的多个序列来优化自己的行动策略，以最大化得到的奖励。

序列决策的挑战在于**信用分配问题**（credit assignment problem）。当智能体获得某个奖励时，应该将这个奖励归功于之前的哪些行动？由于行动的效果可能会延迟显现，智能体需要学会识别哪些行动对最终结果产生了积极或消极的影响。

(2) 状态转换与马尔可夫假设

马尔可夫假设是强化学习理论基础的核心。它简化了环境动态的建模复杂性，使得我们可以专注于当前状态而不必考虑整个历史序列。

状态是对环境当前情况的完整描述，它包含了智能体做出最优决策所需的所有相关信息。如果不考虑智能体的行动和收益，环境的**状态转换**过程可以表示为：

$$P(s_{t+1}|s_1, s_2, \dots, s_t)$$

该式表明，下一个环境状态的概率分布依赖于所有历史状态。这虽然与事实相符合，但这种依赖关系在计算上是难以处理的。因为需要考虑的状态历史组合是指数级的。

马尔可夫假设将这种依赖关系简化为：

$$P(s_{t+1}|s_1, s_2, \dots, s_t) = P(s_{t+1}|s_t)$$

这意味着，给定当前状态 s_t ，未来状态 s_{t+1} 的概率分布与历史状态 $s_1, s_2, \dots, s_{t-1}$ 无关。这种“无记忆”性质大大简化了状态转移的建模和计算。

马尔可夫假设的有效性取决于状态表示的充分性。如果状态包含了所有决策相关的信息，那么马尔可夫假设通常是成立的。在实际应用中，如果原始观察不满足马尔可夫性，我们可以通过扩展状态表示（例如，包含历史信息）来近似满足这个假设。

(3) 状态转移矩阵与马尔可夫链

在离散状态空间中，环境状态的转换可以用**状态转移矩阵**来表示。设状态空间 $S = \{s_1, s_2, \dots, s_N\}$ ，状态转移矩阵 $\mathbf{P}$ 是一个 $N \times N$ 的矩阵，其中元素 $P_{ij} = P(s_j|s_i)$ 表示从状态 s_i 转移到状态 s_j 的概率。

$$\mathbf{P} = \begin{pmatrix} p(s_1|s_1) & p(s_2|s_1) & \dots & p(s_N|s_1) \\ p(s_1|s_2) & p(s_2|s_2) & \dots & p(s_N|s_2) \\ \vdots & \vdots & \ddots & \vdots \\ p(s_1|s_N) & p(s_2|s_N) & \dots & p(s_N|s_N) \end{pmatrix}$$

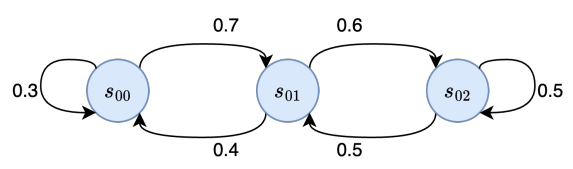
状态转移矩阵具有以下重要性质：

- 每行元素之和为1: $\sum_{j=1}^N P_{ij} = 1$ ，这反映了概率的归一化性质。
- 所有元素都是非负的: $P_{ij} \geq 0$ ，符合概率的基本要求。
- 矩阵的幂 $\mathbf{P}^n$ 表示经过 n 步转移的概率，即 $(\mathbf{P}^n)_{ij} = P(S_{t+n} = s_j | S_t = s_i)$ 。

考虑一个简单的三状态系统，其状态转移矩阵为：

$$\mathbf{P} = \begin{pmatrix} 0.3 & 0.7 & 0 \\ 0.4 & 0 & 0.6 \\ 0 & 0.5 & 0.5 \end{pmatrix}$$

接下来，我们分别利用 s_{00}, s_{01}, s_{02} 表示这三个状态。从这个转移矩阵中可以知道，从状态 s_{00} 出发有70%的概率转移到状态 s_{01} ，有30%的概率保持在状态 s_{00} ；从状态 s_{01} 出发，有60%的概率转移到状态 s_{02} ，有40%的概率转移到状态 s_{00} ；从状态 s_{02} 出发，有50%的概率转移到状态 s_{01} ，有50%的概率保持在状态 s_{02} ，有20%的概率保持在状态 s_{03} 。这个状态转换过程还可以用下图表示：



对这个状态转移过程所描述的环境进行观测，可能得到如下轨迹：

$s_{00}, s_{01}, s_{02}, s_{01}, s_{02}, s_{02}$
 $s_{00}, s_{00}, s_{01}, s_{00}, s_{01}, s_{02}, s_{01}$
 $s_{00}, s_{01}, s_{00}, s_{00}, s_{01}, s_{00}$

依赖于相同参数（时间）的一组随机变量的序列称为**随机过程**（Stochastic Process）。满足马尔可夫性质的随机过程称为**马尔可夫过程**或**马尔可夫链**。在离散时间、离散状态的情况下，马尔可夫链可以完全由其状态空间和状态转移矩阵来描述。

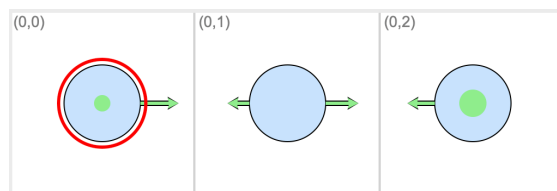
马尔可夫链的重要性在于长期行为的平稳性。如果马尔可夫链是非周期的且所有状态都是相互可达的（即不可约的），那么系统所处状态的概率分布将收敛到一个唯一的平稳分布 S ，满足：

$$S = S\mathbf{P}$$

平稳分布表示系统在长期运行后各状态的概率分布，它不再随时间变化。这个概念在强化学习中具有重要意义，因为它描述了在给定策略下，智能体访问各状态的长期频率。

- **例：**重新将上述状态转移矩阵表示为如下表格，并利用GridWorld实现这个过程，如下图所示。

	s_{00}	s_{01}	s_{02}
s_{00}	0.3	0.7	0
s_{01}	0.4	0	0.6
s_{02}	0	0.5	0.5



- 代码 05-markov_1.py 中，以任意状态为初始状态，接下来根据状态转移矩阵随机转移当前状态。经过迭代，各状态出现的频率收敛致 $[0.20, 0.36, 0.43]$ 附近。
- 代码 05-markov_2.py 随机初始化 S ，接下来令 $S = SP$ ，经过迭代后， S 同样收敛于 $[0.20, 0.36, 0.43]$ 附近

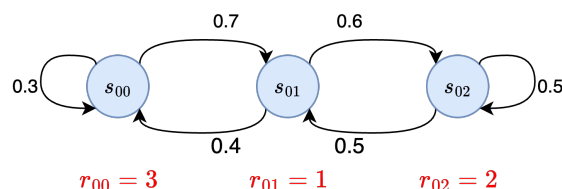
2.3 奖励与状态价值函数

(1) 奖励与回报

如果只考虑环境状态的变化，那么Agent就当相于河面上随波逐流的小船，它既不会改变环境也不能感知环境。

奖励（Reward）是在状态转换的过程中，Agent从每个状态中得到的收益，可以是正数（表示好的行为）、负数（表示不好的行为）或零（表示中性行为）。奖励使得Agent具有环境感知能力，是强化学习中的核心概念，它为智能体的学习提供了指导信号。通过奖励机制，环境向智能体传达了什么样的行为是期望的，什么样的行为应该避免。

状态奖励函数描述给定状态 s 下，Agent能够获取的即时奖励，表示为 $R(s)$ 。从时间的维度来看，我们也常将Agent在 t 时刻获取的即时奖励表示为 r_t 。如下图所示的状态转移过程中，在每个状态下（或每个时刻）Agent都能够得到不同的奖励。



即时奖励只反映了Agent在单个状态或单个时刻的收益，而强化学习关注的是长期的累积收益，即**回报**（return）。它定义为从某个时间点开始的累积奖励：

$$G_t = r_{t+1} + r_{t+2} + r_{t+3} + \dots$$

回报的概念具有预测性，因为在应用实际中它是指 t 时刻Agent未来预期总收益。因此，时间间隔越久奖励的不确定性就越大，未来的奖励应该比即时奖励具有较低的权重。这可以通过在回报中引入**折扣因子** $\gamma \in [0, 1]$ 来描述：

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

当 $\gamma = 0$ 时，智能体只关心即时奖励；当 $\gamma = 1$ 时，所有未来奖励都被同等对待；当 γ 接近1时，智能体更加重视长期收益。折扣因子的作用有多个方面：

- 数学收敛性：确保在无限时间范围内回报是有限的。

- 不确定性建模：反映对未来预测的不确定性。
- 即时性偏好：体现对即时奖励的偏好。
- 计算简化：使得后续价值函数的计算更加稳定。

(2) 马尔可夫奖励过程

在马尔可夫链的基础上，如果我们在状态转移的过程中引入奖励（reward），就得到了**马尔可夫奖励过程**（Markov Reward Process, MRP）。

一个马尔可夫奖励过程由一个四元组定义：

$$\mathcal{M} = (\mathcal{S}, \mathbf{P}, R, \gamma)$$

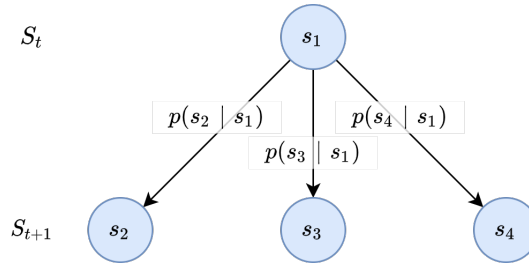
其中：

- $\mathcal{S}$ ：状态空间，表示环境可能处于的所有状态集合；
- $\mathbf{P}$ ：状态转移矩阵， $\mathbf{P}_{ij} = P(s_j | s_i)$ 表示从状态 s_i 转移到状态 s_j 的概率；
- R ：状态奖励函数，定义在状态空间上， $R(s) = \mathbb{E}[R_{t+1} | S_t = s]$ 表示智能体在状态 s 下的期望奖励；
- $\gamma \in [0, 1]$ ：折扣因子，用于平衡即时奖励与未来奖励的重要性。

(3) 状态价值函数及其估计方法

在马尔可夫奖励过程中，我们用**状态价值函数**刻画了某个状态的“好坏程度”。即从 t 时刻状态 s 开始，未来能够获得的累积奖励期望：

$$V(s) = \mathbb{E}[G_t | S_t = s]$$



如图所示， t 时刻的状态 $S_t = s_1$ ，在 $t + 1$ 时刻的状态 S_{t+1} 可能是 s_2, s_3, s_4 。利用状态转移矩阵， s_1 的状态价值可计算为：

$$\begin{aligned} V(s_1) &= R(s_1) \\ &+ \gamma p(S_{t+1} = s_2 | S_t = s_1) V(s_2) \\ &+ \gamma p(S_{t+1} = s_3 | S_t = s_1) V(s_3) \\ &+ \gamma p(S_{t+1} = s_4 | S_t = s_1) V(s_4) \end{aligned}$$

需要注意的是，上式中包含了 $R(s_1)$ ，这是因为按照约定**奖励并非产生在状态发生期间，而是在转移至新状态时才产生的**。上式也可以简写为：

$$V(s_1) = R(s_1) + \gamma p(s_2 | s_1) V(s_2) + \gamma p(s_3 | s_1) V(s_3) + \gamma p(s_4 | s_1) V(s_4)$$

上式的一般形式表示称为马尔可夫奖励过程的**贝尔曼期望方程**（Bellman Expectation Equation）：

$$V(s) = R(S_t = s) + \gamma \sum_{s'} p(S_{t+1} = s' | S_t = s) V(S_{t+1} = s')$$

或简写为：

$$V(s) = R(s) + \gamma \sum_{s'} p(s'|s) V(s')$$

在离散状态的情况下，我们可以将所有状态的奖励函数表示为一个向量。该向量满足：

$$\begin{pmatrix} V(s_1) \\ V(s_2) \\ \vdots \\ V(s_N) \end{pmatrix} = \begin{pmatrix} R(s_1) \\ R(s_2) \\ \vdots \\ R(s_N) \end{pmatrix} + \gamma \begin{pmatrix} p(s_1 | s_1) & p(s_2 | s_1) & \dots & p(s_N | s_1) \\ p(s_1 | s_2) & p(s_2 | s_2) & \dots & p(s_N | s_2) \\ \vdots & \vdots & \ddots & \vdots \\ p(s_1 | s_N) & p(s_2 | s_N) & \dots & p(s_N | s_N) \end{pmatrix} \begin{pmatrix} V(s_1) \\ V(s_2) \\ \vdots \\ V(s_N) \end{pmatrix}$$

该式是所有状态下贝尔曼期望方程的组合，表示为矩阵形式：

$$\mathbf{V} = \mathbf{R} + \gamma \mathbf{P} \mathbf{V}$$

其中， $\mathbf{R}$ 是各状态下的即时奖励， $\mathbf{P}$ 为状态转移方程。

上述过程实际上给出了马尔可夫奖励过程状态价值函数的估计方法。当状态转移矩阵已知时，我们可以直接求解线性方程得：

$$\mathbf{V} = (\mathbf{I} - \gamma \mathbf{P})^{-1} \mathbf{R}$$

也可以根据马尔可夫过程的平稳性，通过迭代更新 $V(s) \leftarrow R(s) + \gamma \sum_{s'} P(s'|s) V(s')$ 直致收敛。这种方法得到的是状态价值函数的精确解。

当状态转移矩阵未知时（环境不可完全观测），我们可以使用蒙特卡罗法估计状态价值。包括如下步骤：

- 从 $S_t = s$ 开始，生成多个轨迹；
- 计算每条轨迹中 s 的折扣奖励；
- 对所有轨迹的折扣奖励取均值。

• 例：

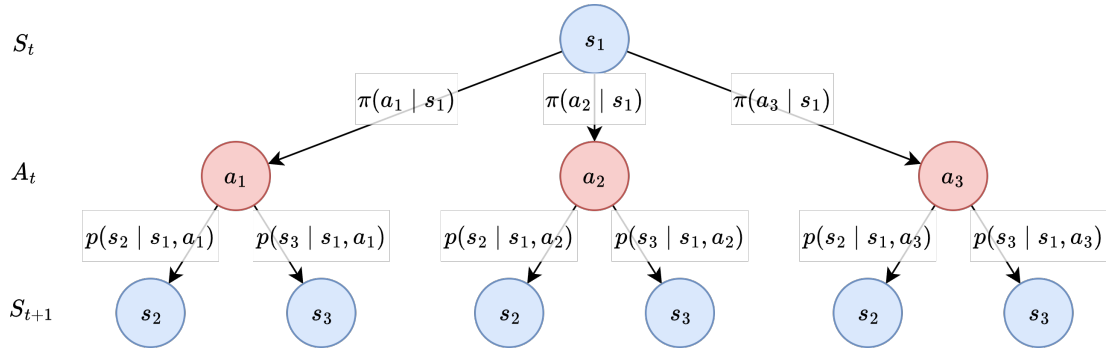
- 代码 06-markov_reward_1.py 利用蒙特卡罗法来估计前一小节例子中三个状态的价值。
- 代码 06-markov_reward_2 利用迭代贝尔曼期望方程的方法来估计。

2.4 策略与动作价值函数

(1) 策略

奖励的加入使得 Agent 能够感知到环境的反馈，但马尔可夫奖励过程中的 Agent 依旧像是随波逐流的小船上的乘客，虽然能够欣赏岸边的风景但是不能控制船的走向。因此，我们需要进一步引入**动作**（Action）使得 Agent 能够根据环境的变化做出响应。这样，就构成了完整的强化学习过程。

在强化学习中，**策略**（Policy）是智能体在给定状态下选择动作的规则。也就是说，动作由策略生成，策略刻画了智能体的行为方式，是从状态到动作的映射。引入策略和动作之后，状态的变化会受到动作的影响，不再仅仅依赖于固定的状态转移概率。动作依赖于策略，因此策略影响着状态的变化。引入策略和行动后的状态转移如下图所示。



根据生成动作中的随机性，可以将策略分为两种类型：

- **确定性策略**：在每个状态 s 下，策略给出唯一的动作：

$$\pi(s) = a$$

表示在状态 s 下必然选择动作 a 。

- **随机性策略**：在每个状态 s 下，策略给出一个概率分布：

$$\pi(a | s) = P(A_t = a | S_t = s)$$

表示在状态 s 下选择动作 a 的概率。

当采用**确定性策略**时，智能体会基于当前的知识，始终选择它认为能够带来**最高预期回报**的行动，这被称为**利用（exploitation）**。而当策略中引入一定的随机性，使智能体有机会尝试那些**尚不确定或回报未知**的行动时，这种行为被称为**探索（exploration）**。在实际应用中，智能体往往需要在探索与利用之间保持平衡，以既能获取足够的信息，又能实现较高的回报。

策略 π 是强化学习的核心对象，所有的价值函数都以**给定某一策略**的前提下定义的。例如，在引入策略之后，**状态价值函数**记为 $V^\pi(s)$ ，表示在状态 s 下遵循策略 π 的预期回报：

$$V^\pi(s) = \mathbb{E}^\pi[G_t | S_t = s] = \mathbb{E}^\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \middle| S_t = s \right]$$

(2) 马尔可夫决策过程

马尔可夫决策过程（Markov Decision Process, MDP）就是引入了动作的马尔可夫奖励过程，由如下五元组定义：

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathbf{P}, R, \gamma)$$

其中：

- $\mathcal{S}$ ：状态空间，表示环境可能处于的所有状态集合；

- $\mathcal{A}$: 动作空间，表示智能体可采取的所有动作集合；
- $\mathbf{P}$: 状态转移概率，表示在状态 s 采取动作 a 后转移到状态 s' 的概率：

$$\mathbf{P}(s' | s, a) = P(S_{t+1} = s' | S_t = s, A_t = a)$$

- R : 奖励函数，表示在状态 s 下采取动作 a 后的期望奖励：

$$R(s, a) = \mathbb{E}[R_{t+1} | S_t = s, A_t = a]$$

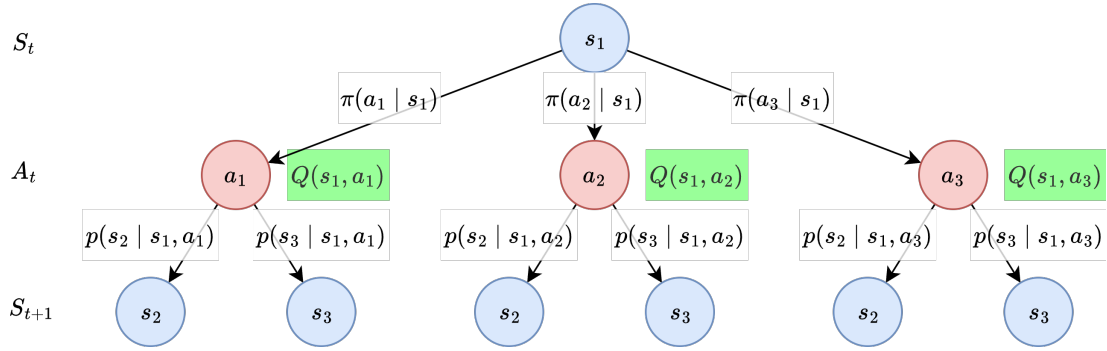
- $\gamma \in [0, 1]$: 折扣因子。

(3) 动作价值函数

在给定策略 π 的情况下，定义智能体在状态 s 下采取动作 a 后，未来能够获得的期望回报称为**动作价值函数** (action-value function)：

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a]$$

动作价值函数在决策过程中的位置如下图所示。



与状态价值函数 $V^\pi(s)$ 不同，动作价值函数不仅考虑了当前所处的状态，还考虑了当前采取的动作。二者关系为：

$$V^\pi(s) = \sum_a \pi(a | s) Q^\pi(s, a)$$

进一步地，我们还可以得到：

$$Q^\pi(s, a) = R(s, a) + \gamma \sum_{s'} p(s' | s, a) V^\pi(s')$$

其中， $R(s, a)$ 是执行动作 a 在状态 s 后获得的**期望即时奖励**：

$$R(s, a) = \mathbb{E}[r_{t+1} | s_t = s, a_t = a]$$

在某些情况下，奖励还可能依赖于下一个状态 ($R(s, a, s')$)，则有：

$$R(s, a) = \sum_{s'} p(s' | s, a) R(s, a, s')$$

马尔可夫决策过程中，状态价值函数满足新的贝尔曼期望方程：

$$V^{\pi}(s) = \sum_a \pi(a|s) \left(R(s, a) + \gamma \sum_{s'} p(s'|s, a) V^{\pi}(s') \right)$$

类似地，关于动作价值函数的贝尔曼期望方程为：

$$Q^{\pi}(s, a) = R(s, a) + \gamma \sum_{s'} p(s'|s, a) \sum_{a'} \pi(a'|s') Q^{\pi}(s', a')$$

从动作价值函数中，我们可直接得到**在某个状态下选择某个动作的好坏程度**，它是Q-learning、SARSA等经典强化学习算法的核心。

(4) 最优策略与贝尔曼最优方程

在强化学习的核心目标中，我们不仅仅关心某个具体策略的价值，而是要找到**最优策略** π^* ，使得智能体在长期回报上达到最大化。

首先，从状态价值的角度来看，在状态 s 下智能体从该状态出发并按照最优策略行动时能够获得的最大期望回报，称为**最优状态价值函数**：

$$V^*(s) = \max_{\pi} V^{\pi}(s)$$

类似地，在状态 s 下选择动作 a ，然后再按照最优策略行动时能够获得的最大期望回报，称为**最优动作价值函数**：

$$Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a)$$

一旦我们得到了最优动作价值函数 $Q^*(s, a)$ ，就可以直接导出**最优策略** π^* ：

$$\pi^*(s) = \arg \max_a Q^*(s, a)$$

即在每个状态下选择使得 $Q^*(s, a)$ 最大的动作。

由最优状态价值函数和最优动作价值函数，我们可得到贝尔曼最优方程：

• 状态价值函数形式：

$$V^*(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} p(s'|s, a) V^*(s') \right]$$

• 动作价值函数形式：

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} p(s'|s, a) \max_{a'} Q^*(s', a')$$

从形式上看，贝尔曼最优方程与贝尔曼期望方程的区别在于：贝尔曼期望方程针对的是某一固定策略 π ，它通过当前奖励加上下一个状态在该策略下的期望价值来定义当前状态的价值；而贝尔曼最优方程则用最大化运算替代期望运算在，用“选择能带来最大后续价值的动作”来替代“按照策略取平均”，从而得到最优价值函数 V^*, Q^* 的递推关系。换句话说，贝尔曼期望方程说明在给定策略下价值如何由一步一步的决策累积得到，而贝尔曼最优方程说明最优价值函数是如何由每一步的最优选择逐层构建出来的。贝尔曼最优方程为许多经典强化学习算法提供了理论基础，例如 **Q-learning** 等。

3. 策略评估与策略控制

策略评估和策略控制是强化学习中的两个核心问题。**策略评估**关注如何准确估计给定策略下的价值函数，而**策略控制**则致力于寻找最优策略。这两个问题的解决方法构成了强化学习算法的基础。

3.1 策略评估

策略评估的目标是计算给定策略 π 下的状态价值函数 $V^\pi(s)$ 或行动价值函数 $Q^\pi(s, a)$ 。根据对环境模型的依赖程度和计算方式的不同，策略评估可以分为多种方法。这与马尔可替过程中状态价值的估计一致。

实际上，代码 `06-markov_reward_1.py` 和 `06-markov_reward_2` 就是在进行策略评估，状态转移的方式实际上就是策略。需要注意的是，`GridWorld`是一个简易的环境，奖励仅依赖于状态 s 。

(1) 动态规划法

动态规划是一种基于模型的策略评估方法，它假设环境的状态转移概率 $P(s'|s, a)$ 和奖励函数 $R(s, a)$ 是已知的。动态规划方法利用贝尔曼期望方程的递归结构来迭代计算价值函数。

其策略评估的迭代更新规则为（贝尔曼期望方程）：

$$V_{k+1}(s) = \sum_a \pi(a|s) \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V_k(s') \right]$$

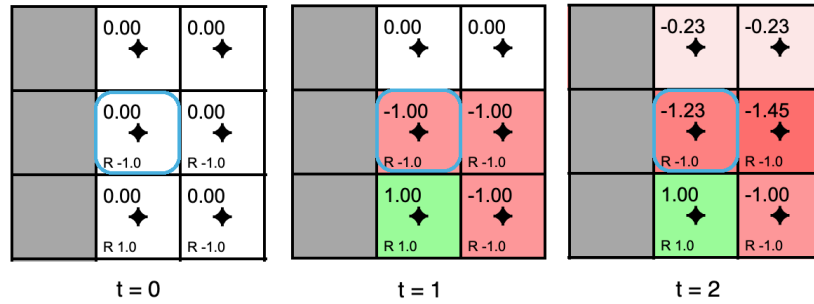
这个迭代过程从某个初始价值函数 V_0 开始，通过反复应用贝尔曼期望方程来更新价值估计。在适当的条件下（如折扣因子 $\gamma < 1$ ），这个迭代序列会收敛到真实的价值函数 V^π 。

如下图所示的例子中，每一个格子表示一个状态。格子中上部数字表示状态价值，下部数字表示奖励。假定当前采取这样的策略：当前状态转向相邻某状态和留在当前状态具有相同的概率。在 $t = 1$ 时刻，蓝框标出的状态价值的更新为：

$$\begin{aligned} V_1(s) &= 0.25 \times [-1 + 0.9 \cdot 0] + 0.25 \times [-1 + 0.9 \cdot 0] \\ &\quad + 0.25 \times [-1 + 0.9 \cdot 0] + 0.25 \times [-1 + 0.9 \cdot 0] \\ &= -1 \end{aligned}$$

其他状态的价值也做相应的更新。在 $t = 2$ 时刻，同一状态价值的更新表示为：

$$\begin{aligned} V_2(s) &= 0.25 \times [-1 + 0.9 \cdot 0] + 0.25 \times [-1 + 0.9 \cdot -1] \\ &\quad + 0.25 \times [-1 + 0.9 \cdot -1] + 0.25 \times [-1 + 0.9 \cdot 1] \\ &= -1.225 \end{aligned}$$



- 例: 07-policy_evaluate_1.py

动态规划方法的优势在于其理论保证和快速收敛性。在有限状态空间中，该方法具有多项式时间复杂度。然而，动态规划需要完整的环境模型，这在许多实际应用中是不现实的。

(2) 蒙特卡罗法

蒙特卡罗方法是一种无模型的策略评估技术，它通过采样完整的轨迹来估计价值函数。该方法的基本思想是利用经验平均来逼近期望值。

对于状态价值函数的估计，蒙特卡罗方法的步骤如下：

- 使用策略 π 生成大量轨迹
- 对于每个轨迹中的每个状态 s ，计算从该状态开始的实际回报 G_t
- 对所有访问状态 s 的回报取平均值作为 $V^\pi(s)$ 的估计

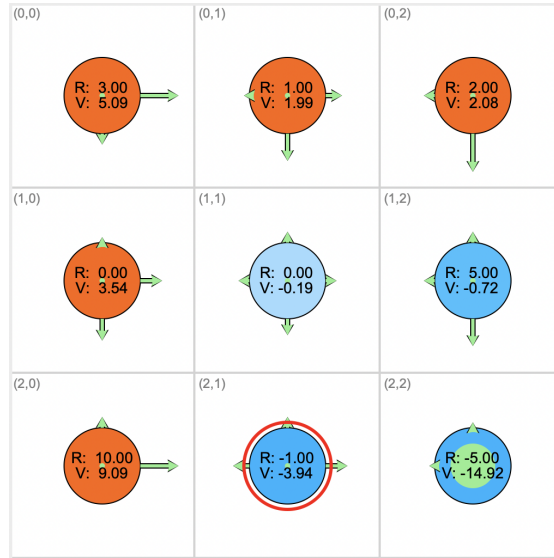
蒙特卡罗估计可以表示为：

$$V^\pi(s) \approx \frac{1}{N(s)} \sum_{i=1}^{N(s)} G_t^{(i)}$$

其中 $N(s)$ 是状态 s 被访问的总次数， $G_t^{(i)}$ 是第 i 次访问状态 s 时的回报。

蒙特卡罗方法可以分为首次访问蒙特卡罗和每次访问蒙特卡罗。**首次访问**蒙特卡罗只在轨迹中第一次访问某状态时更新其价值估计，而**每次访问**蒙特卡罗在每次访问时都进行更新。两种方法在理论上都会收敛到正确的价值函数。

- 例: 07-policy_evaluate_2.py



蒙特卡罗方法的一个重要优势是其无偏性。由于 $E[G_t | S_t = s] = V^\pi(s)$ ，蒙特卡罗估计是真实价值函数的无偏估计。然而，该方法的方差通常较大，需要大量样本才能获得准确的估计。

为了提高计算效率，增量更新方法提供了一种不用保存所有历史回报、直接在每次得到新样本时逐步修正估计的实现方式。这种方法是蒙特卡罗估计在计算机实现上的一种递推公式，它与蒙特卡罗在数学上等价，但更节省存储、支持在线学习，因此常作为蒙特卡罗方法的实际更新手段。

假设有对状态 s 的 n 个回报样本 $G_1, \dots, G_n$ ，则样本平均为

$$V_n^\pi(s) = \frac{1}{n} \sum_{i=1}^n G_i$$

加入第 $n+1$ 个样本 G_{n+1} 之后有：

$$V_{n+1}^\pi(s) = \frac{1}{n+1} \sum_{i=1}^{n+1} G_i = \frac{1}{n+1} (n \cdot V_n^\pi(s) + G_{n+1})$$

展开并整理可得到：

$$V_{n+1}^\pi(s) = V_n^\pi(s) + \frac{1}{n+1} (G_{n+1} - V_n^\pi(s))$$

由于 $V_n^\pi(s)$ 和 V_{n+1}^π 是历史均值，因此有如下的迭代更新规则：

$$V^\pi(s) \leftarrow V^\pi(s) + \frac{1}{n+1} (G_{n+1} - V_n^\pi(s))$$

在实际实现中，我们通常会将 $\frac{1}{n+1}$ 替换为一个固定的更新权重，并且将 G_{n+1} 简写为 G ，于是可得到增量更新规则：

$$V^\pi(s) \leftarrow V^\pi(s) + \alpha [G - V^\pi(s)]$$

- 例：07-policy_evaluate_3.py

(3) 时序差分法

动态规划和蒙特卡罗两种方法各有优缺点：

- **动态规划 (DP)**：利用环境的转移概率和奖励函数，基于贝尔曼期望方程更新价值函数。
 - 优点：更新及时（可在理论上直接求解或迭代求解价值）。
 - 缺点：需要知道完整的环境模型（转移概率和奖励），在很多实际问题中不可得。
- **蒙特卡罗方法 (MC)**：通过完整的一条轨迹，用真实回报 $G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+1+k}$ 来更新价值函数。
 - 优点：估计无偏（直接使用实际回报）。
 - 缺点：必须等到回合结束才能更新，方差较大，不适合持续型任务或需要低延迟更新的场景。

时序差分方法 (Temporal Difference, TD) 结合了两者的优点——不依赖环境的完整模型（model-free），并且可以在每一步进行更新。其核心思想是**自举 (bootstrapping)**：用当前的价值估计来改进价值估计，而不是等待完整回合。

一方面，根据回报的定义可知，从时刻 $t+1$ 开始的真实回报为：

$$G_{t+1} = r_{t+2} + \gamma r_{t+3} + \dots$$

另一方面，根据定义，**状态价值函数 $V^\pi(s_{t+1})$ 是未来回报 G_{t+1} 的期望**：

$$V^\pi(s_{t+1}) = \mathbb{E}[G_{t+1} \mid S_{t+1} = s_{t+1}]$$

如果我们不想等到 G_{t+1} 完整的未来结果出来，就可以直接用 $V^\pi(s_{t+1})$ 来替代它。 $V^\pi(s_{t+1})$ 虽然不是精确的回报，但在期望意义下是正确的目标。随着不断学习， V^π 会越来越接近真实值。

于是，可以得到最简单的时序差分方法 TD(0)，更新规则为：

$$V^\pi(s_t) \leftarrow V^\pi(s_t) + \alpha [R_{t+1} + \gamma V^\pi(s_{t+1}) - V^\pi(s_t)],$$

其中方括号内为**时序差分误差 (TD error)**， $R_{t+1} + \gamma V^\pi(s_{t+1})$ 是对 G_t 的一步估计。

时序差分方法能够在每一步交互后立即更新价值估计，而不必等到整个轨迹结束，因此它能在与环境交互的过程中实现在线学习。由于每一步的计算量为常数，因而效率很高。虽然引入自举会带来偏差，但通常能够显著降低方差，从而在有限样本下收敛更快。此外，时序差分方法不依赖完整回合，因此能自然处理持续型任务，在实践中比蒙特卡罗方法更常用。

(4) n 步时序差分

TD(0)和蒙特卡罗方法可以看作是 n 步时序差分方法的特殊情况。 n 步TD方法使用 n 步回报来更新价值估计：

$$G_t^{(n)} = r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{n-1} r_{t+n} + \gamma^n V^\pi(s_{t+n})$$

相应的更新规则为：

$$V^\pi(s_t) \leftarrow V^\pi(s_t) + \alpha [G_t^{(n)} - V^\pi(s_t)]$$

将 $G_t^{(n)}$ 代入：

$$V^\pi(s_t) \leftarrow V^\pi(s_t) + \alpha [r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{n-1} r_{t+n} + \gamma^n V^\pi(s_{t+n}) - V^\pi(s_t)]$$

当 $n=1$ 时，退化为 TD(0)；当 $n=\infty$ 时，它就等价于蒙特卡罗方法。

通过调节 n 的大小，可以在偏差与方差之间进行平衡：较小的 n 引入较多偏差但方差更低，较大的 n 偏差更小但方差更大。 n 步时序差分提供了一种灵活的方式，在两类方法之间取得折衷。

- 例：08-policy_evaluate_td.py

(5) 策略评估方法比较

动态规划方法依赖于完整的环境模型，通过递推计算即可在理论上保证快速收敛，因此在模型已知且状态空间相对较小的情形下非常高效。然而，一旦环境模型不可完全观测或状态空间过大，动态规划的应用就会受到限制。

蒙特卡罗方法则不需要环境模型，它通过采样完整的轨迹并对回报取平均来估计价值函数。在理论上，这种估计是无偏的，但往往存在方差较大的问题，导致收敛速度较慢。蒙特卡罗方法主要适用于情节性任务，尤其是在状态转移关系复杂、难以显式建模时。

时序差分方法在一定程度上结合了动态规划与蒙特卡罗的优势。它既不依赖完整的环境模型，又可以在与环境交互的过程中逐步更新价值函数，从而实现在线学习。由于每一步的计算量是常数级的，因此在计算效率上具有明显优势。此外，TD方法通过自举更新降低了估计的方差，但其估计在初期通常带有一定偏差。总体而言，TD方法能够自然地处理连续任务，是应用最为广泛的策略评估方法之一。

方法	优点	缺点	适用场景
动态规划 (DP)	收敛速度快；理论收敛性有保证	需要完整的环境模型；状态空间过大时难以计算	模型已知且状态空间较小的问题
蒙特卡罗 (MC)	不需要环境模型；估计无偏	方差较大；收敛速度较慢；必须等到回合结束才能更新	情节性任务，尤其是状态转移复杂、难以建模的环境
时序差分 (TD)	不需要环境模型；可以在线学习；计算效率高；方差较小；可处理连续任务	估计在初期存在偏差；理论分析较复杂	大多数实际强化学习问题，特别是连续任务

3.2 策略控制

策略控制的目标是寻找最优策略 π^* ，使得价值函数最大化。策略控制问题的解决通常基于策略评估和策略改进的交替进行，形成一个不断优化的过程。根据对环境模型的依赖程度和算法实现方式的不同，策略控制可以分为多种方法。

(1) 策略迭代

策略迭代 (Policy Iteration) 是一种基于模型的策略控制方法，它将最优策略搜索问题分解为两个互相依赖的步骤：策略评估和策略改进。这种方法的核心思想是通过反复交替执行这两个步骤，逐步逼近最优策略。

- **策略评估步骤**：计算当前策略 π_k 的精确价值函数 V^{π_k} 。可以使用动态规划方法通过求解以下线性方程组来完成：

$$V^{\pi_k}(s) = \sum_a \pi_k(a|s) \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V^{\pi_k}(s')].$$

- **策略改进步骤**：基于当前价值函数构造一个更好的策略。新策略 π_{k+1} 采用贪心策略：

$$\pi_{k+1}(s) = \arg \max_a \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V^{\pi_k}(s')]$$

这种贪心策略改进的理论基础是策略改进定理。

策略改进定理： 设 π 和 π' 是两个确定性策略，如果对所有状态 s 都有：

$$Q^\pi(s, \pi'(s)) \geq V^\pi(s)$$

那么策略 π' 至少与策略 π 一样好，即对所有状态 s 都有 $V^{\pi'}(s) \geq V^\pi(s)$ 。

证明： 定义策略 ρ_n 为：在前 n 步使用 π' ，第 $n+1$ 步起改用 π 。显然 $\rho_0 = \pi$ ，且 $\lim_{n \rightarrow \infty} \rho_n = \pi'$ 。

- 当 $n = 1$ 时

$$V^{\rho_1}(s) = \mathbb{E}[R_{t+1} + \gamma V^\pi(S_{t+1}) \mid S_t = s, A_t = \pi'(s)] = Q^\pi(s, \pi'(s)) \geq V^\pi(s) = V^{\rho_0}(s).$$

- 假设对于所有状态 s 都有 $V^{\rho_n}(s) \geq V^{\rho_{n-1}}(s)$ ，则

$$V^{\rho_{n+1}}(s) - V^{\rho_n}(s) = \gamma, \mathbb{E}[V^{\rho_n}(S_{t+1}) - V^{\rho_{n-1}}(S_{t+1}) \mid S_t = s, A_t = \pi'(s)] \geq 0,$$

因为条件期望的被积函数在各后继状态上非负（归纳假设）。由归纳得对所有 n 和所有 s ， $V^{\rho_n}(s)$ 随 n 单调不减。取极限（在折扣且奖励有界的情形下允许交换极限与期望等操作），得到 $V^{\pi'}(s) \geq V^\pi(s)$ 。■

直观地，每增加一步用新策略 π' 代替旧策略 π ，我们都只是在当前状态选择一个**不会比原动作更差的动作**。由于每一步的期望回报至少和原策略一样好，因此前 $n+1$ 步的总期望回报不会低于前 n 步的回报。换句话说，每一次“替换动作”的操作都是单调非减的，所以整个价值函数序列随着前缀长度增加而单调上升，最终得到完全使用 π' 的策略，其期望回报至少与原策略 π 相同。

想象我们在玩一个游戏，每走一步都可以选择动作。我们有两种策略：旧的 π 和新的 π' 。策略改进定理的意思是，如果在每个状态下， π' 的动作至少和 π 一样好，那么我们可以把“按 π' 走前 n 步，再按 π 走后面”想象成一条路径。每多换一步动作为 π' ，得到的总分不会比少换一步时低——因为每一步都不会变糟。随着换的步数越来越多，最终整条路径都是 π' 的动作，总分自然不会比完全用旧策略低。换句话说，**一步步换动作，回报不会下降，最终整个策略的表现至少和原来一样好。**

策略迭代算法的伪代码如下：

输入：MDP (S, A, P, R, γ) ，收敛阈值 θ

1. 初始化：

对所有状态 s ： $V(s) \leftarrow$ 任意初值

对所有状态 s ： $\pi(s) \leftarrow$ 任意动作

2. 策略评估：

repeat:

$\Delta \leftarrow 0$

 for 每个状态 $s \in S$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

```
until  $\Delta < \theta$ 
```

3. 策略改进:

```
policy_stable  $\leftarrow$  True  
for 每个状态  $s \in S$ :  
    old_action  $\leftarrow \pi(s)$   
     $\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s'} P(s'|s,a) [R(s,a) + \gamma V(s')]$   
    if old_action  $\neq \pi(s)$ :  
        policy_stable  $\leftarrow$  False
```

4. 如果 policy_stable = True, 则停止并返回 $V \approx V^*$, $\pi \approx \pi^*$; 否则转到步骤 2

- 例: 09-policy_iteration.py

(2) 价值迭代

价值迭代 (Value Iteration) 是另一种基于模型的策略控制方法, 它将策略评估和策略改进合并为一个步骤, 直接迭代贝尔曼最优方程:

$$V_{k+1}(s) = \max_a \sum_{s'} P(s'|s,a) [R(s,a) + \gamma V_k(s')]$$

价值迭代可以看作是**截断的策略迭代**, 其中策略评估只进行一次更新, 然后立即进行隐式的策略改进。这种方法不需要显式地维护策略, 而是直接更新价值函数。

价值迭代的优势在于其**简单性和鲁棒性**。它不需要显式的策略表示, 直接操作价值函数。一旦价值函数收敛, 最优策略可以通过贪心选择提取:

$$\pi^*(s) = \operatorname{argmax}_a \sum_{s'} P(s'|s,a) [R(s,a) + \gamma V^*(s')]$$

价值迭代的伪代码如下:

输入: MDP (S, A, P, R, γ), 收敛阈值 θ

1. 初始化:

对所有状态 s : $V(s) \leftarrow$ 任意初值

2. 价值迭代:

```
repeat:  
     $\Delta \leftarrow 0$   
    for 每个状态  $s \in S$ :  
         $v \leftarrow V(s)$   
         $V(s) \leftarrow \max_a \sum_{s'} P(s'|s,a) [R(s,a) + \gamma V(s')]$   
         $\Delta \leftarrow \max(\Delta, |v - V(s)|)$   
until  $\Delta < \theta$ 
```


3. 输出确定性最优策略：

```
for 每个状态  $s \in S$ :  
     $\pi^*(s) \leftarrow \operatorname{argmax}_a \sum_{s'} P(s'|s,a) [R(s,a) + \gamma V(s')]$ 
```

- 例：10-value_iteration.py

(3) 策略迭代和价值迭代比较

从整体上看，**策略迭代**和**价值迭代**都属于基于模型的动态规划方法，它们的核心思想都是通过贝尔曼最优方程的迭代求解来逼近最优价值函数与最优策略。但两者在实现方式上存在显著差异：

• 更新方式

- 策略迭代将过程分为两个明确的步骤：策略评估（通常是精确或近似地计算 (V^{π}) ）和策略改进（根据 (V^{π}) 贪心更新策略）。每次迭代都可能需要较多的计算来完成策略评估。
- 价值迭代则将这两个步骤合并为一个：每次直接用贝尔曼最优算子更新价值函数，即“部分评估 + 立即改进”，避免了完整的策略评估过程。

• 收敛速度

- 策略迭代在每次评估步骤中代价较高，但通常只需要很少的迭代次数就能收敛到最优策略。
- 价值迭代每次更新代价较小，但需要更多次的迭代才能使价值函数收敛。

• 策略表示

- 策略迭代在迭代过程中始终显式维护一个策略 (π) 。
- 价值迭代则不直接存储策略，而是通过价值函数的贪心选择隐式定义当前策略，在收敛后再显式提取最优策略。

• 本质联系，策略迭代和价值迭代其实可以看作是广义策略迭代（GPI）的两个极端：

- 策略迭代中，策略评估是“完全评估”，策略改进是“完全贪心”。
- 价值迭代中，策略评估是“只做一步更新”，然后立刻改进。
- 介于二者之间的“截断策略迭代”就是在评估过程中只做有限次迭代，再进行改进。

因此，在实际应用中，两者的选择往往取决于问题规模与计算资源：小规模 MDP 中，策略迭代通常更高效；而在大规模或需要近似计算的场景下，价值迭代及其近似变体（如 Q-learning）更为常见。

(4) 广义策略迭代

在策略迭代与价值迭代中，我们都可以看到**策略评估与策略改进的交替过程**。事实上，这种“评估—改进”的思想并不限于完全精确的评估或严格的贪心改进。在实践中，我们往往只做**部分的策略评估**（例如只迭代几步贝尔曼更新，而不是完全收敛），或者采用**近似的策略改进**（例如使用 ϵ -贪心而非纯贪心）。这种更一般的形式被称为**广义策略迭代（Generalized Policy Iteration, GPI）**。

广义策略迭代强调的并不是某个具体算法，而是一种普遍框架：**只要评估过程推动价值函数向当前策略的真实值逼近，而改进过程推动策略向价值函数的贪心方向调整，二者交替的动态相互作用最终就会收敛到最优策略**。换句话说，策略迭代和价值迭代都可以看作是广义策略迭代的特例。

4. 同策略与异策略学习

在强化学习中，根据学习过程中使用的策略与目标策略的关系，可以将算法分为同策略（On-policy）和异策略（Off-policy）两大类。这种分类对理解和设计强化学习算法具有重要意义。

4.1 基本概念

在强化学习中，关于“策略”有两个重要的概念需要进行区分：

- **目标策略**（Target Policy）是智能体最终希望学习和优化的策略，它决定了在学习完成后智能体如何选择动作以最大化长期累积奖励。
- **行为策略**（Behavior Policy）是智能体在学习过程中实际采用的策略，用于与环境交互并收集经验数据。

也就是说，目标策略强调“学成之后应该如何行动”，而行为策略强调“在学习过程中如何获取数据”。当目标策略与行为策略为相同的策略时，称为**同策略学习**（on-policy learning）；当目标策略与行为策略不同时，称为**异策略学习**（off-policy learning）。

同策略算法中，智能体既使用该策略与环境交互，又依赖同一策略进行优化。由于数据分布与优化目标完全一致，这类方法在理论分析和实践应用中都相对稳定直观。

与之相比，异策略学习（off-policy learning）允许目标策略与行为策略不同。智能体可以通过一种策略来收集经验数据，同时基于另一种策略进行学习。这种方式虽然在实现上更为复杂，但却提供了更大的灵活性：

- **探索效率**：可以使用专门的探索策略来收集数据，而不影响目标策略的学习；
- **数据重用**：可以重复使用历史数据，提高样本效率；
- **安全学习**：可以使用安全的行为策略来收集数据，避免危险的探索；
- **多任务学习**：可以同时学习多个不同的目标策略；
- **模仿学习**：可以从其他智能体（如人类专家）的行为中学习。

4.2 SARSA算法

SARSA（State-Action-Reward-State-Action）是一种典型的同策略算法，它直接学习行动价值函数 $Q^\pi(s, a)$ 。

(1) Q表格方法

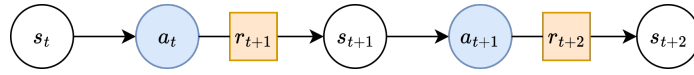
SARSA 以时序差分（TD）思想对动作价值函数 $Q(s, a)$ 进行逐步更新。在状态空间与动作空间有限的情况下，常用一个大小为 $|S| \times |A|$ 的 Q 表来显式存储每个状态—动作对的估计值，其中元素 $Q(s, a)$ 表示在状态 s 下采取动作 a 所期望的折扣累积回报。

	a_1	a_2	$\cdots$	a_{n-1}	a_n
s_1	$q_{1,1}$		$\cdots$		
s_2			$\cdots$		
s_3			$\cdots$		
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$
s_m			$\cdots$		$q_{m,n}$

对于任一固定策略 π ，动作价值函数满足贝尔曼期望方程

$$Q^\pi(s, a) = R(s, a) + \gamma \sum_{s'} \left(p(s'|s, a) \sum_{a'} \pi(a'|s') Q^\pi(s', a') \right)$$

智能体与环境的交互可产生一个序列 $s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}, r_{t+2}, s_{t+2}, \dots$ ，如下图所示。



将序列中的 $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$ 看作一个样本，并采用增量式 TD 更新：

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[\underbrace{r_{t+1} + \gamma Q(s_{t+1}, a_{t+1})}_{\text{target}} - \underbrace{Q(s_t, a_t)}_{\text{current}} \right]$$

式中仅需要数据 $s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}$ ，因此这种方法称为SARSA。

SARSA更新式中的 a_{t+1} 是在状态 s_{t+1} 上依据当前策略 π （即行为策略与目标策略一致）采样得到的动作，因此 SARSA 属于**同策略（on-policy）TD 方法**，TD 误差为：

$$\delta_t = r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t),$$

该量刻画了当前估计与基于下一步经验构造的目标值之间的差异，学习率 α 控制每次更新的步长。在满足充分探索与适当的步长衰减条件下、使用查表表示的 SARSA 可收敛到对应策略的动作价值函数。若状态或动作空间过大，则需以函数逼近替代显式 Q 表以保证可行性。

(2) 基于状态价值的方法与基于动作价值方法比较

在强化学习中，基于状态价值的方法与基于 Q 表格的方法在研究对象和应用方式上有所不同。状态价值方法的核心在于学习状态价值函数 $V^\pi(s)$ ，它衡量了智能体在状态 (s) 下出发，并在后续遵循策略 π 时所能获得的期望累积回报。相比之下，Q 表格方法则直接学习动作价值函数 $Q^\pi(s, a)$ ，它不仅依赖于所处状态，还显式刻画了在该状态下选择特定动作的长期价值。

这种差异导致了二者在决策过程中的不同角色。若仅有状态价值函数，智能体在决策时往往需要结合环境的转移概率模型 $P(s'|s, a)$ ，以便判断在给定状态下执行某一动作是否能带来更高的未来回报。相对而言，**若已知动作价值函数，则可以通过简单的贪婪选择 $\arg \max_a Q(s, a)$ 来直接确定最优动作，而无需借助环境模型。**这也是 Q 表格方法在未知环境下更加实用的原因。

两类方法虽然研究对象不同，却都建立在贝尔曼方程的统一框架之上。状态价值函数满足递归关系

$$V^\pi(s) = \mathbb{E}^\pi[r_{t+1} + \gamma V^\pi(s_{t+1}) \mid s_t],$$

而动作价值函数则满足

$$Q^\pi(s, a) = \mathbb{E}^\pi[r_{t+1} + \gamma Q^\pi(s_{t+1}, a_{t+1}) \mid s_t, a_t].$$

由此可以看出，二者在形式上完全平行，只是研究对象的刻画粒度不同。如果已知 Q 值，就能够通过

$$V^\pi(s) = \sum_a \pi(a|s) Q^\pi(s, a)$$

得到状态价值函数；反之，若掌握了 $V^\pi(s)$ 以及环境转移模型，则同样可以推导出对应的 Q 值。

在控制问题中，二者也体现出不同的使用场景。基于状态价值的方法常见于策略迭代与价值迭代等模型驱动型方法，其优势在于计算与分析简洁；而基于 Q 表格的方法则更适用于环境动力学未知的情境，因为它能够在交互过程中直接学习到可用于行动决策的动作价值估计，从而导出最优策略。

状态价值方法与 Q 表格方法的区别主要体现在研究对象和决策依赖性上，但它们同属贝尔曼方程的理论框架，并能够在一定条件下相互转化。二者最终目标一致，都是为了获得能够指导智能体实现长期最优回报的策略。

(3) 探索策略 (ε-贪婪)

在强化学习中，智能体需要在“利用” (exploitation) 已有知识和“探索” (exploration) 未知环境之间保持平衡。过度利用可能导致陷入局部最优，而过度探索则会降低学习效率。SARSA 等方法常采用 **ε-贪婪 (ε-greedy) 策略** 来实现这种平衡。

ε-贪婪策略的基本思想是：大部分情况下，智能体依据当前的价值函数选择最优动作，但仍以一定概率选择随机动作，以保证探索的持续进行。其概率分布可形式化地写作：

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|A|}, & \text{当 } a = \arg \max_{a'} Q(s, a') \\ \frac{\epsilon}{|A|}, & \text{其他情况} \end{cases}$$

其中 $\epsilon \in [0, 1]$ 用于控制探索程度。当 $\epsilon = 0$ 时，策略完全贪婪，只选择当前估计最优的动作；当 $\epsilon = 1$ 时，策略退化为完全随机选择。通常的设定是在二者之间取一个较小的值，使智能体既能集中利用已有知识，又能以一定概率跳出局部模式、尝试新的选择。

在实际应用中，常常采用随时间逐渐递减的探索率，以便在学习初期鼓励探索，在后期趋向稳定利用。一种常见的做法是设定

$$\epsilon_t = \max(\epsilon_{\min}, \epsilon_0 \cdot \lambda^t),$$

其中 ϵ_0 为初始探索率， $\lambda \in (0, 1)$ 为衰减因子， $\epsilon_{\min}$ 则确保探索率不会降低到零。这种方式使得智能体在训练过程中能够逐步由探索主导转向利用主导，从而兼顾学习效率与策略收敛性。

(4) 算法实现

完整的SARSA算法可以描述如下：

算法：SARSA

- 初始化 $Q(s, a)$ 对所有 $s \in \mathcal{S}, a \in \mathcal{A}$ (除终止状态外)

- 对每个episode:
 - 初始化状态 S
 - 根据 Q 选择动作 A （使用 ϵ -贪婪策略）
 - 对episode中的每一步:
 - 执行动作 A ，观察奖励 R 和新状态 S'
 - 根据 Q 在状态 S' 中选择动作 A' （使用 ϵ -贪婪策略）
 - 更新: $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)]$
 - $s \leftarrow s'; a \leftarrow a'$
 - 直到 s 是终止状态

SARSA算法的收敛性可以通过随机逼近理论来证明。在适当的条件下（如学习率满足 $\sum_t \alpha_t = \infty$ 且 $\sum_t \alpha_t^2 < \infty$ ，以及充分的探索），SARSA会以概率1收敛到最优Q函数。

- 例: 11-sarsa.py

4.3 Q-Learning 算法

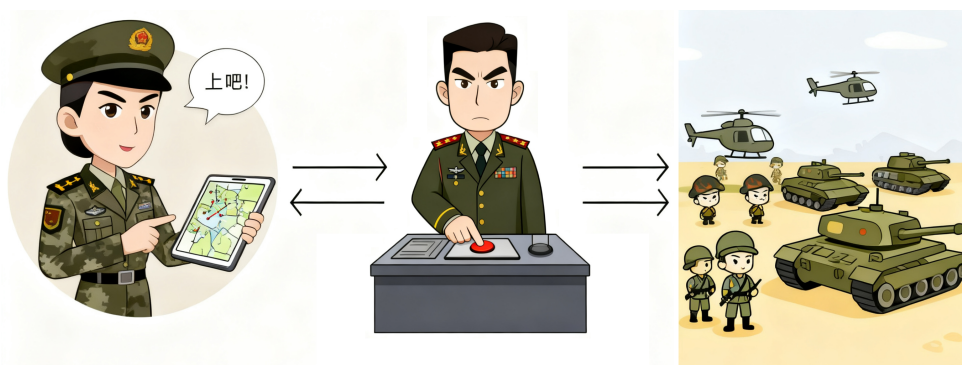
Q-Learning 是一种经典的异策略（off-policy）强化学习算法。其核心思想是直接学习最优动作价值函数 $Q^*(s, a)$ ，从而能够在行为策略与目标策略不一致的情况下，依然保证收敛到最优策略。

(1) 从同策略到异策略

在同策略（on-policy）方法（如 SARSA）中，智能体必须使用当前执行策略进行价值估计，这带来两个主要问题：

- 如果行为策略包含探索（如 ϵ -贪婪策略），则学习到的价值函数会包含探索带来的偏差，难以收敛至最优；
- 策略更新严重依赖于采样轨迹，数据利用效率不高。

为解决上述问题，我们希望设计一种算法，即使采样策略与目标策略不同，也能准确学习最优价值函数。Q-Learning 正是基于这一动机提出的。



(2) 基本原理

Q-Learning 是基于对贝尔曼最优方程的近似。贝尔曼最优方程给出了最优动作价值函数 $Q^*(s, a)$ 的递归关系：

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} p(s'|s, a) \max_{a'} Q^*(s', a')$$

Q-Learning 采用时序差分方法，通过迭代方式逼近该方程，其更新规则为：

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right].$$

Q-Learning 的关键在于：

- 更新目标为 $r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a')$ ，即假设在下一状态 s_{t+1} 中选取最优动作；
- 该目标与当前实际执行的动作无关，因此学习过程独立于行为策略，直接面向最优策略。这种设计正是 Q-Learning 异策略特性的来源。

Watkins 和 Dayan（1992）给出了 Q-Learning 的收敛性定理：在有限马尔可夫决策过程（MDP）中，若满足以下条件：

- 每个状态-动作对 (s, a) 被无限次访问；
- 学习率 α_t 满足：

$$\sum_{t=0}^{\infty} \alpha_t = \infty, \quad \sum_{t=0}^{\infty} \alpha_t^2 < \infty,$$

则 Q-Learning 算法以概率 1 收敛到最优动作价值函数 Q^* 。

(3) 算法描述

Q-Learning 的基本算法流程如下：

算法：Q-Learning（表格法）

- 初始化：对所有 $s \in \mathcal{S}$ ， $a \in \mathcal{A}$ ，初始化 $Q(s, a)$ ；
- 对每一条轨迹（episode）重复：
 - 初始化状态 s ；
 - 每步循环：
 1. 根据行为策略（如 ϵ -贪婪策略）根据状态 s 选择动作 a ；
 2. 执行动作 a ，观察奖励 r 与下一状态 s' ；
 3. 更新 Q 值：

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right];$$

4. 更新状态： $S \leftarrow s'$ ；

- 直到 s 为终止状态。

注意：行为策略通常使用 ϵ -贪婪策略以保证充分探索。

(4) Q-Learning 与 SARSA 的对比

Q-Learning 与 SARSA 的比较如下表所示。两者最主要的区别在于 SARSA** 强调“学习当前执行的”，而Q-Learning 强调“学习最优的，无论你当前执行什么”。

	SARSA	Q-Learning
更新目标	$r_{t+1} + \gamma Q(s_{t+1}, a_{t+1})$ (基于实际执行的动作)	$r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a')$ (基于最优动作)
策略类型	同策略 (on-policy)	异策略 (off-policy)
收敛目标	收敛至当前策略 π 对应的 Q^π	收敛至最优动作价值函数 Q^*
风险倾向	保守，考虑探索带来的不确定性	激进，假设下一步总能选择最优动作
适用场景	适用于对实际执行策略进行评估，如需要安全探索的任务	适用于直接学习最优策略，如仿真环境或高样本效率场景

Q-Learning 作为一种经典的异策略时序差分控制算法，其核心在于能够直接学习并收敛到最优动作价值函数 Q^* 。这一目标的实现，得益于其更新目标与智能体实际执行的行为策略无关，因此它可以在诸如 ϵ -贪婪之类的任意探索策略下进行稳定学习。与同策略的SARSA算法相比，Q-Learning的学习目标更为积极，理论上具有更强的收敛性，但这也导致其在样本有限时可能因过度乐观而显得不够稳定。尽管如此，Q-Learning以其简洁的形式和坚实的理论基础，成为了连接传统强化学习与深度强化学习的桥梁，为后续深度Q网络（DQN）等突破性算法的出现奠定了基石。

5. 深度Q学习

传统的Q-Learning算法虽然在理论上具有良好性质，但在实际应用中面临着严重的可扩展性问题。当状态空间或动作空间变得庞大时，Q表格的存储和更新变得不可行。深度Q学习（Deep Q-Learning, DQN）通过引入深度神经网络来近似Q函数，突破了传统方法的限制，开启了深度强化学习的新时代。

5.1 从Q表格到深度Q网络

(1) Q表格的局限性

在传统Q-Learning算法中，Q函数通常以表格（Q表格）的形式存储。Q表格通过一个二维数组记录每个状态-动作对的价值估计。如果状态空间为 $\mathcal{S}$ ，动作空间为 $\mathcal{A}$ ，则完整的Q表需要存储 $|\mathcal{S}| \times |\mathcal{A}|$ 个条目。然而，在许多实际问题中，状态空间可能是高维的，甚至是连续的。例如，在Atari游戏任务中，智能体的观测状态往往是 84×84 的图像。如果每个像素可以取256个灰度值，那么可能状态数目将达到 $256^{84 \times 84}$ ，这个数量远远超出计算机可存储与处理的范围。这就是Q-Learning中的“维度灾难”。

更为严重的是，即便我们能够存储所有状态-动作对，Q表格的学习效率也极为低下。由于每一个状态都被视为独立的实体，即使两个状态几乎完全相似，它们对应的Q值也必须通过单独的交互经验来学习。这意味着Q表格无法利用状态之间的相似性进行知识迁移，从而导致训练样本需求量呈指数级增长。此外，Q表格方法也无法直接处理连续的状态空间。通常的做法是先将连续状态离散化，但这不仅会导致精度损失，还会引发新的维度爆炸问题。

- **维度灾难**：当状态空间大小为 $|\mathcal{S}|$ 、动作空间大小为 $|\mathcal{A}|$ 时，Q表格需要 $|\mathcal{S}| \times |\mathcal{A}|$ 个存储单元。
- **连续状态处理能力不足**：Q表格只能处理离散状态空间。对于连续状态空间，需要进行离散化处理，这会导致信息损失和维数爆炸。
- **泛化能力缺失**：Q表格无法利用状态之间的相似性。即使两个状态非常相似，它们的Q值也需要独立学习，无法实现知识迁移。
- **样本利用率低下**：每个状态-动作对都需要足够的访问次数才能获得准确的价值估计，这在大状态空间中需要极大的样本量。

(2) 神经网络作为Q函数逼近器

深度Q学习的核心思想是使用参数化的函数逼近器来替代Q表格。具体而言，我们用一个神经网络 $Q(s, a; \theta)$ 近似Q函数，其中 θ 表示网络的参数。与表格方法相比，神经网络具有多方面的优势。

- **紧凑表示**：神经网络能够提供紧凑的表示。它可以通过相对有限的参数刻画高维输入与动作之间的复杂关系，从而显著减少存储需求。
- **泛化能力**：神经网络具有强大的泛化能力。对于未见过的状态，网络可以根据输入特征与已有经验推断出合理的Q值估计，实现跨状态的知识迁移。
- **连续性处理**：神经网络天然支持连续输入，无需进行显式的离散化处理，这使其能够直接处理连续状态空间的问题。
- **特征学习**：深度神经网络具有自动特征学习的能力。在复杂环境中（例如图像输入），它能够自行从原始像素中提取多层次的抽象特征，避免了手工设计特征的繁琐和局限。

因此，使用神经网络逼近Q函数不仅是解决维度灾难的现实需求，也是强化学习与深度学习结合的关键所在。

5.2 深度Q网络的结构设计

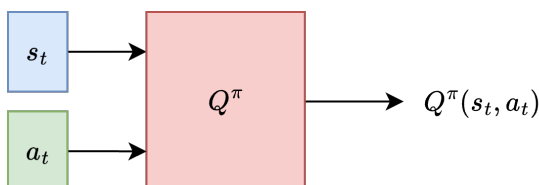
在深度强化学习中，深度Q网络（Deep Q-Network, DQN）以神经网络作为函数逼近器来近似动作价值函数 $Q^\pi(s, a)$ ，从而使得在高维状态空间中仍能进行价值估计与决策。在实际实现上，按照网络输入/输出的不同，常见的结构设计可分为三类：一是以状态和动作为输入、直接输出Q值的结构；二是以状态为输入、一次性输出所有动作对应的Q值的结构（典型DQN）；三是以状态为输入、输出状态价值V的结构（通常用于评估/作为Critic）。

(1) 神经网络评估状态行动组合的价值

这种设计方式与Q-Learning的理论一致。网络被看作映射：

$$f_\theta(s, a) = Q(s, a; \theta)$$

即把状态-动作对 (s, a) 作为输入，直接输出该对的动作价值。此类结构在动作是连续或高维时很有价值，因为网络可以接受任意（连续）动作向量作为输入。



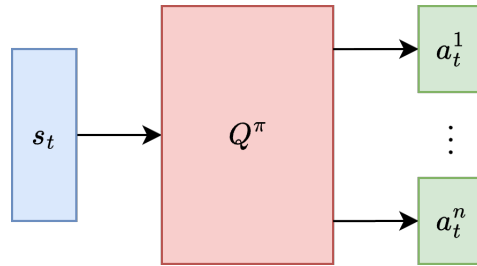
尽管理论上可行，但这种结构在动作选择时存在困难。Q-Learning的“贪婪选择”要求计算 $\arg \max_a Q(s, a; \theta)$ 。当动作空间是离散且可枚举时可以直接枚举；但当动作空间连续或维度大时，求该最大值变成连续优化问题，求解困难。尽管如此，这种结构设计在与策略梯度方法相结合时有很重要的应用价值，例如DDPG算法。

(2) 神经网络枚举行动价值

这是经典DQN中最常见的结构。网络的输入为状态 s ，输出为所有动作集合 $\{a_1, \dots, a_n\}$ 的状态价值：

$$f_{\theta}(s) = [Q(s, a_t^1; \theta), Q(s, a_t^2; \theta), \dots, Q(s, a_t^n; \theta)]^T$$

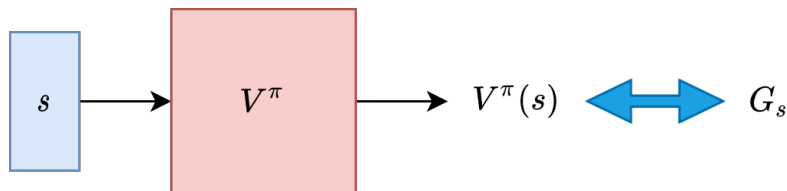
对于离散动作集，这种做法在计算效率和实现简洁性上具有明显优势：决策只需取向量的最大分量，选择复杂度仅为一次前向传播加一个 $\arg \max$ 操作。



该结构的优点在于可直接比较所有离散动作的价值，极大提升了决策速度，适用于需要实时响应的场景（如 Atari、许多离散动作控制任务）。然而，它固有的局限是仅适合动作集合固定且可枚举的离散情形；当动作空间很大时，输出头的维度随之增长，网络参数与样本复杂度都会显著增加，从而带来过拟合与学习难度。

(3) 神经网络评估状态价值

也可以用神经网络来拟合状态价值函数 $f_{\theta}(s) = V^{\pi}(s)$ ，对状态价值进行评估。在纯价值评估（policy evaluation）的问题中，可用 TD(0) 的目标 $y_t = r_{t+1} + \gamma V(s_{t+1}; \theta)$ 或蒙特卡洛回报来训练。



这种设计方式的典型用途不是独立地做控制，而是作为 Actor-Critic 框架中的 Critic（评估器）。在 Actor-Critic 中，Critic 提供关于动作优劣的评估信号，常以优势函数 $A(s, a)$ 作为连接 Actor 与 Critic 的中介：

$$A(s, a) = Q(s, a) - V(s)$$

(4) 比较分析

从理论和工程两个角度来看，网络结构选择应由以下关键因素共同决定：动作空间的类型（离散或连续）、对决策延迟的允许程度、样本效率要求、以及能否并行实现动作优化。

若动作集合为离散且规模适中，第二种“状态输入、输出全动作Q值”的设计是首选：它实现简单、决策快速，并且与稳定化技术结合时在实践中已被广泛验证。若动作为连续或非常高维，第一种“状态+动作输入的 Critic + actor”组合则更为合适。若目标是纯评估或需要低方差的基线估计，则采用第三种“状态价值 $V(s; \theta)$ ”作为 Critic 更为稳健，且在 Actor-Critic 与 A2C/GAE 等方法中被广泛采用。

5.3 Deep Q-Learning 算法基本原理

深度Q学习的核心在于利用神经网络逼近Q函数，并在此基础上延续Q-Learning的时序差分思想。与传统Q-Learning相同，算法的目标是逼近最优动作价值函数 $Q^*(s, a)$ ，使得在任意状态下智能体都能够选择具有最高长期回报的动作。由于Q函数由神经网络 $Q(s, a; \theta)$ 表示，学习过程依赖于梯度下降来最小化合适的损失函数。

(1) 时序差分估计

在Q-Learning中，更新规则基于时序差分（Temporal-Difference, TD）方法。传统Q表格更新方式为：

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right],$$

其中括号内的差值被称为TD误差（TD error），它衡量了当前Q值估计与目标回报之间的偏差。

在深度Q学习中，Q函数由参数化的神经网络 $Q(s, a; \theta)$ 表示，因此无法像表格方法那样直接更新某个条目，而是需要构造监督信号，并通过梯度下降来更新参数。具体而言，给定样本 $(s_t, a_t, r_{t+1}, s_{t+1})$ ，TD目标（target）定义为：

$$y_t = r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a'; \theta),$$

TD误差表示为：

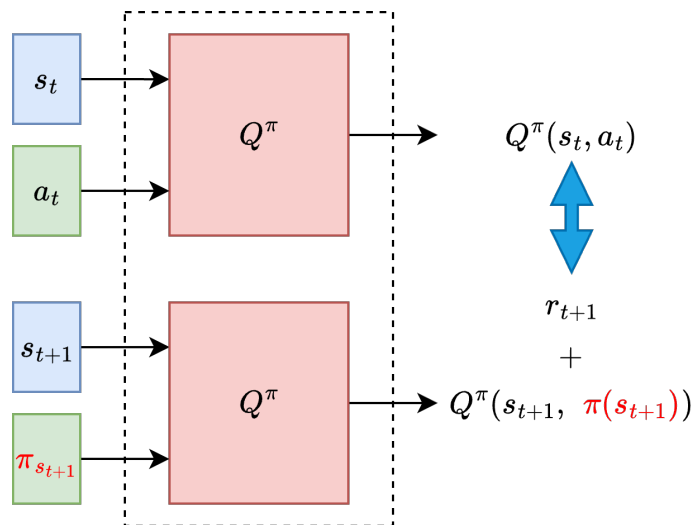
$$\delta_t = y_t - Q(s_t, a_t; \theta).$$

通过最小化TD误差的平方，神经网络得以逐步逼近最优Q函数。

具体实现中，给定如下轨迹：



行动价值的时序差分估计的计算表示为：



其中， $\pi(s_{t+1}) = \arg \max_{a_{t+1}} Q(s_{t+1}, a_{t+1})$ 表示 $t + 1$ 时刻的使得动作价值最大化的动作。

(2) 损失函数

为了利用梯度下降更新参数，需要将TD目标转化为损失函数的形式。深度Q学习的标准损失函数定义为

$$\mathcal{L}(\theta) = \mathbb{E}_{(s_t, a_t, r_{t+1}, s_{t+1})} \left[(y_t - Q(s_t, a_t; \theta))^2 \right]$$

其中期望是对样本分布取平均。在实际实现中，通常采用小批量样本来近似该期望，从而得到一个经验损失函数。

对该损失函数关于参数 θ 求梯度，可以得到反向传播所需的更新方向。这表明深度Q学习本质上是通过监督学习的方式来逼近TD目标值。损失函数的最小化使得网络输出逐渐收敛到真实的动作价值估计。

需要注意的是，TD目标 y_t 依赖于网络自身的估计，因此训练过程可能会陷入不稳定。为缓解这一问题，深度Q学习通常在实践中采用一个“目标网络”，并且在若干步更新之后才将训练网络的参数复制给目标网络，以保证目标值在短时间内相对稳定。

(3) 探索策略

在DQN中，探索策略依然是不可或缺的环节。如果只依赖贪婪动作选择，网络很容易陷入局部最优解，因此必须在利用和探索之间保持平衡。最常见的方式是 ϵ -贪婪策略，其定义为

$$a = \begin{cases} \arg \max_{a'} Q(s, a'; \theta), & \text{以概率 } 1 - \epsilon \\ \text{随机选择动作}, & \text{以概率 } \epsilon \end{cases}$$

其中参数 ϵ 控制探索强度，通常随训练进程逐渐衰减。

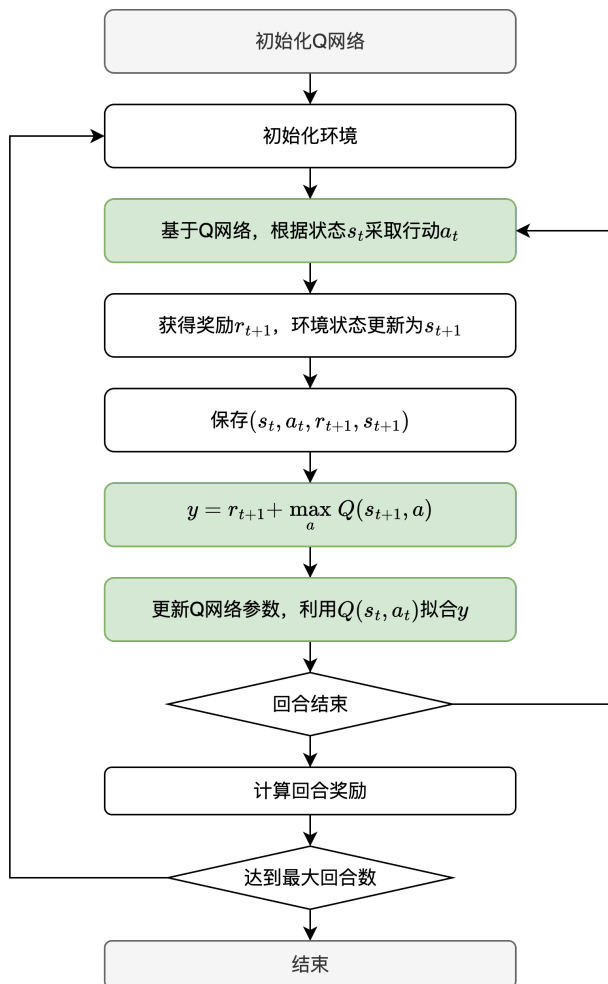
另一种方式是基于Boltzmann分布的随机策略，即根据各个动作的Q值分配概率：

$$P(a|s) = \frac{\exp(Q(s, a)/\tau)}{\sum_{a'} \exp(Q(s, a')/\tau)},$$

其中温度参数 τ 决定探索程度：当 τ 较大时，各动作概率趋近均匀分布；当 τ 较小时，动作选择更偏向于高Q值的动作。除了这两种经典方式，近年来也有在神经网络参数中引入噪声的探索方法（如NoisyNet），通过在权重中加入可学习的噪声实现更具结构化的探索行为。

(4) 算法流程

综合上述内容，深度Q学习的算法流程可以描述如下。首先初始化参数化的Q网络 $Q(s, a; \theta)$ 以及对应的目标网络 $Q(s, a; \hat{\theta})$ ，并设定 $\hat{\theta} = \theta$ 。在每个时间步中，智能体根据 ϵ -贪婪策略选择动作，与环境交互并获得新的状态与即时奖励。随后，利用经验样本构造TD目标 y_t ，并通过最小化损失函数来更新网络参数。



(5) 有效性与理论解释

深度Q学习虽然在实践中表现卓越，但其理论分析要比表格型Q-Learning复杂得多。这主要源于函数逼近的引入，使得更新过程不再具有严格的收敛保证。尽管如此，我们依然可以从三个角度理解其有效性。

首先是逼近误差的角度。设最优Q函数为 $Q^*(s, a)$ ，而神经网络的最优逼近为 $Q(s, a; \theta^*)$ ，则二者之间的误差可以记为

$$\epsilon(s, a) = Q^*(s, a) - Q(s, a; \theta^*)。$$

只要这种逼近误差在可控范围内，算法的性能损失就能够被限制在一定程度内。

其次是收敛性角度。传统Q-Learning在有限状态空间和满足探索条件的情况下可以收敛到最优Q函数。而对于DQN而言，由于目标值依赖于网络参数本身，导致更新目标具有非平稳性，严格收敛性难以保证。但在网络容量足够大且采用合适稳定化技术（如目标网络与经验回放）的情况下，DQN能够在实践中逼近最优解。

最后是泛化性的角度。神经网络的强大表达能力和跨状态的泛化能力，使得它在有限的训练样本下依然能够对未见过的状态给出合理的Q值估计。这种泛化特性正是深度学习结合强化学习能够突破传统方法瓶颈的核心原因。

5.4 DQN算法优化

尽管深度Q学习在高维任务（如Atari游戏）中取得了突破性成功，但原始的DQN训练往往极不稳定，甚至可能完全失效。造成这一现象的主要原因在于目标函数的非平稳性、样本序列的高度相关性，以及Q值估计过程中的偏差和方差问题。因此，为了保证算法的可训练性和稳定性，研究者提出了一系列关键的改进方法，其中最核心的便是引入目标网络（Target Network）和经验回放（Experience Replay）。

(1) 目标网络

在标准DQN中，用于计算目标值的Q网络和正在被更新的Q网络参数完全相同，即

$$y_t = r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a'; \theta_t)。$$

然而，由于 θ_t 在训练过程中不断变化，这意味着目标值本身在每次更新时都在波动。这种“移动目标”现象会引入不稳定性，使得学习过程可能陷入震荡甚至发散。

为了解决这一问题，DQN引入了一个目标网络。该网络参数记作 $\hat{\theta}$ ，它与主网络具有相同的结构，但参数在训练过程中保持相对固定，仅在一定间隔之后才从主网络复制一次，或者通过软更新的方式逐渐逼近主网络。此时目标值的计算变为：

$$y_t = r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a'; \hat{\theta})$$

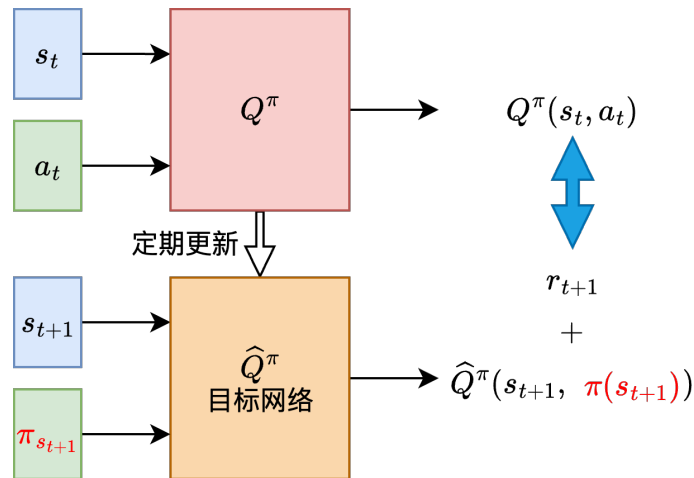
对应的损失函数是：

$$L(\theta_t) = \mathbb{E}_{(s_t, a_t, r_{t+1}, s_{t+1})} \left[(y_t - Q(s_t, a_t; \theta_t))^2 \right]。$$

通过引入目标网络，训练的目标在一段时间内保持稳定，避免了“自举”过程中过度放大的偏差，从而有效提升了训练的稳定性。目标网络采用如下方式定期更新：

$$\hat{\theta} \leftarrow \tau \theta + (1 - \tau) \hat{\theta},$$

其中 $\tau \ll 1$ 是更新率，用于控制目标网络向主网络缓慢靠近的速度。这种方式在实践中表现出更平滑的训练曲线和更好的收敛特性。



其中， $\pi(s_{t+1}) = \arg \max_{a_{t+1}} Q(s_{t+1}, a_{t+1})$ 表示 $t + 1$ 时刻的使得动作价值最大化的动作。

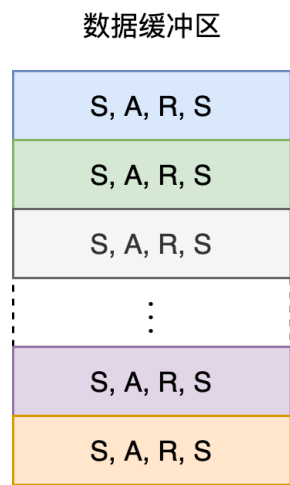
(2) 经验回放机制

在强化学习中，智能体与环境交互得到的状态-动作-奖励序列往往存在强烈的时序相关性。如果直接使用这些相关数据进行梯度更新，网络训练会严重偏离独立同分布（i.i.d.）的假设，容易导致模型陷入局部最优，甚至出现灾难性遗忘。

为了解决这一问题，DQN提出了经验回放机制。具体来说，算法在训练过程中维护一个容量为 N 的经验缓冲区 D ，存储历史交互产生的四元组 $(s_t, a_t, r_{t+1}, s_{t+1})$ 。在更新网络参数时，不再直接使用最近的经验，而是从缓冲区中随机采样一个小批量（minibatch）进行训练。这样做有三个显著好处：

- 通过随机采样打破数据之间的相关性，使得训练过程更接近于i.i.d.假设；
- 显著提高了样本利用率，因为每条经验可以被多次使用；
- 能够结合批量梯度下降的优势，提高计算效率和稳定性。

从理论角度来看，经验回放相当于用经验缓冲区中的分布去近似环境的真实数据分布。在缓冲区足够大且环境变化不剧烈的情况下，这种近似可以较好地维持学习的无偏性，并降低方差。



(3) 完整的DQN流程

结合目标网络与经验回放机制，一个典型的DQN训练流程可以表述如下。首先，初始化一个Q网络 $Q(s, a; \theta)$ ，并复制其参数到目标网络 $Q(s, a; \hat{\theta})$ 。同时，建立一个容量有限的经验缓冲区 D 。在每个回合开始时，智能体初始化状态 s_1 。随后在每个时间步 t ，智能体根据 ϵ -贪婪策略选择动作 a_t ，与环境交互获得奖励 r_t 和下一状态 s_{t+1} ，并将经验 (s_t, a_t, r_t, s_{t+1}) 存入缓冲区。

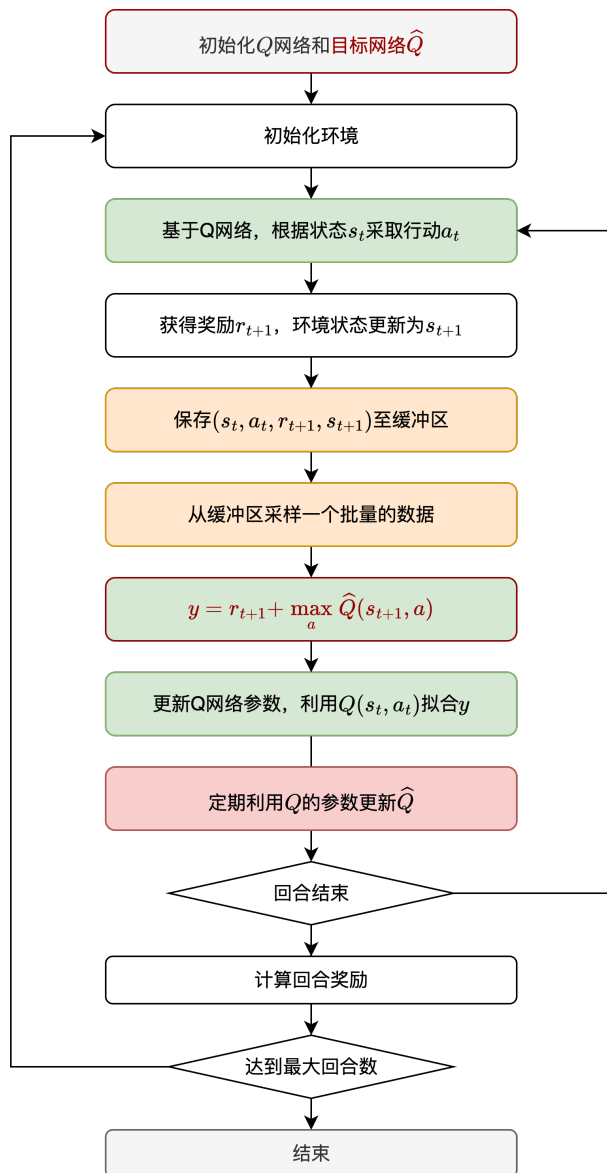
在训练阶段，从 D 中随机采样一个小批量经验 (s_j, a_j, r_j, s_{j+1}) ，计算对应的目标值

$$y_j = \begin{cases} r_j, & \text{若回合在 } j+1 \text{ 时刻终止} \\ r_j + \gamma \max_{a'} Q(s_{j+1}, a'; \hat{\theta}), & \text{否则} \end{cases}$$

然后最小化均方误差损失：

$$L(\theta) = \frac{1}{|\mathcal{B}|} \sum_{j \in \mathcal{B}} (y_j - Q(s_j, a_j; \theta))^2,$$

其中 $\mathcal{B}$ 表示采样的经验批量。通过反向传播和梯度下降更新参数 θ ，并每隔 C 步将 θ 复制到目标网络 $\hat{\theta}$ ，从而保证目标计算的相对稳定性。



这一流程不仅保留了Q-Learning的基本思想，同时通过目标网络和经验回放解决了训练中的核心不稳定因素，使得深度强化学习真正成为可行的方法。正是这两项改进的提出，使得DQN能够在高维环境中展现出远超传统强化学习的表现。

- 例：13-basic_dqn.py

5.5 深度Q学习的改进

深度Q网络（DQN）的提出解决了强化学习在高维状态空间下的函数逼近问题，并成功应用于Atari 2600游戏。然而，原始DQN在训练过程中仍存在一些显著的局限性。例如，目标值估计存在偏差，训练过程不稳定，探索效率不足等。这些问题限制了算法在更复杂环境中的性能。为了克服这些缺陷，研究者们提出了一系列改进方法，包括双重DQN（Double DQN）、决斗DQN（Dueling DQN）以及优先经验回放（Prioritized Experience Replay）。这些方法共同推动了深度Q学习的发展，使其在性能和稳定性上均得到显著提升。

(1) 双重DQN

在原始DQN中，目标值的计算方式为：

$$y_t^{\text{DQN}} = r_t + \gamma \max_{a'} Q(s_{t+1}, a'; \hat{\theta}),$$

其中 $\hat{\theta}$ 表示目标网络的参数。该式存在一个关键问题：同一个Q网络既被用来选择动作，又被用来评估该动作的价值。这种“自举式”的最大化操作往往会带来高估偏差（overestimation bias），尤其是在函数逼近存在噪声时，高估效应会不断累积，影响最终策略质量。

双重DQN（Double DQN）的核心思想是将动作选择与动作评估分离。具体而言，使用当前网络参数 θ 选择动作，使用目标网络参数 $\hat{\theta}$ 对该动作进行估值。其目标值计算公式为：

$$y_t^{\text{Double DQN}} = r_t + \gamma Q(s_{t+1}, \arg \max_{a'} Q(s_{t+1}, a'; \theta); \hat{\theta}).$$

这种方式有效地缓解了高估问题，使得Q值估计更加稳定和接近真实值，从而提升了策略性能。

• 例：14-double_dqn.py

(2) 决斗DQN

在许多强化学习任务中，并非所有状态都需要区分每个动作的价值。例如，在某些状态下，所有动作的效果几乎相同，这时直接学习每个动作的Q值效率较低。决斗DQN（Dueling DQN）通过将Q函数分解为两个部分来解决这一问题：状态价值函数（Value Function）和动作优势函数（Advantage Function）。其基本形式为：

$$Q(s, a; \theta) = V(s; \theta_v) + \left(A(s, a; \theta_A) - \frac{1}{|A|} \sum_{a'} A(s, a'; \theta_A) \right).$$

其中， $V(s; \theta_v)$ 表示在状态 s 下的整体价值， $A(s, a; \theta_A)$ 表示在状态 s 下执行动作 a 相对于平均水平的优势。通过这种分解，网络能够更高效地学习哪些状态本身重要，而不仅仅是动作间的相对差异。这一结构在许多环境中显著提升了学习效率和最终性能。

• 例：15-dueling_dqn.py

(3) 优先经验回放

原始DQN的经验回放机制从回放缓冲区中均匀采样过去的经验。然而，并非所有经验对学习的贡献相同。直观地说，那些预测误差较大的经验更能提供有价值的学习信号。

优先经验回（Prioritized Experience Replay）放基于这一思想，为每条经验分配一个优先级，通常与其时间差分误差（TD Error）的绝对值相关：

$$p_i = |\delta_i| + \epsilon,$$

其中 δ_i 为第 i 条经验的TD误差， ϵ 为防止零概率的常数。采样概率被定义为：

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha},$$

其中 α 控制优先级的强度。当 $\alpha = 0$ 时，采样退化为均匀分布；当 $\alpha > 0$ 时，高误差的经验更容易被采样。

为了避免采样偏差，算法会引入重要性采样权重对梯度进行修正，其形式为：

$$w_i = \left(\frac{1}{N \cdot P(i)} \right)^\beta,$$

其中 N 为回放缓冲区大小， β 用于控制修正程度。通过这种机制，算法能够在保持无偏性的同时，更有效地利用关键经验。

- 例: 16-per_dqn.py

6. 策略梯度方法

SARSA和Q-Learning等算法通过学习状态或状态—动作的价值函数，进而间接地导出最优策略，因此称为以**价值函数**为核心的强化学习方法。然而，当动作空间为连续空间、或者策略本身需要具备随机性时，基于价值的方法会遇到难以克服的局限。例如，在连续控制任务中，动作无法通过简单的“取最大值”方式得到；而在部分可观测环境中，确定性策略往往缺乏稳健性。

为克服这些问题，**策略梯度方法 (Policy Gradient Methods)** 提出了一种直接优化策略的思路。该方法不再依赖显式的价值函数估计，而是通过参数化的策略函数来直接表示智能体的行为决策。通过对期望回报关于策略参数的梯度进行估计和上升更新，智能体可以逐步优化自身的决策模式。这种方法在理论上与基于价值的方法互为补充，适用于连续动作控制与随机策略学习场景。

6.1 基于策略的方法概述

(1) 策略的参数化

在策略梯度方法中，智能体的策略通常用一个带参数的概率分布函数来表示，即

$$\pi(a \mid s; \theta),$$

表示在状态 s 下采取动作 a 的概率， θ 为策略的参数。

这种参数化策略可以由神经网络实现，输入为状态 s ，输出为动作分布的参数（例如均值和方差），从而自然地适应连续动作空间。例如，在机器人控制任务中，策略可以输出一个正态分布：

$$a \sim \mathcal{N}(\mu(s; \theta), \Sigma(s; \theta)),$$

其中 μ 和 Σ 分别由神经网络输出的均值与协方差矩阵表示。智能体的学习目标是通过调整参数 θ ，最大化长期期望奖励。

(2) 与基于价值方法的对比

基于策略的方法与基于价值的方法的主要区别在于**策略的获得方式**。

基于价值的方法（如Q-Learning或DQN）通过近似动作价值函数 $Q^\pi(s, a)$ 或状态价值函数 $V^\pi(s)$ ，再通过贪心策略间接获得决策：

$$\pi(s) = \arg \max_a Q(s, a).$$

这类方法的优势在于直观且样本利用率高，但其适用性受到动作空间维度与连续性的限制。

相比之下，基于策略的方法直接对策略进行建模和优化，而不依赖于“取最大值”运算。其目标是通过梯度上升使期望回报最大化：

$$J(\theta) = \mathbb{E}_{\pi_\theta}[G_t],$$

其中 G_t 表示从时间步 t 开始的回报。策略参数 θ 通过优化目标函数 $J(\theta)$ 不断更新。这种方法天然适合连续动作和随机策略的场景。

从学习性质上看，基于价值的方法倾向于**确定性决策**（即在给定状态下选择一个最优动作），而基于策略的方法则直接学习一个**概率分布**，能够建模策略的不确定性与多样性，这对于复杂环境具有重要意义。

(3) 策略梯度方法的优势与局限

策略梯度方法在强化学习体系中占据重要地位，其优缺点可从理论和实践两方面理解。

从优势来看，首先，它**天然适配连续动作空间**。由于策略函数输出的是动作分布的参数，而非离散动作值，因此在连续控制任务中无需离散化处理，也不必执行复杂的数值优化。其次，策略梯度方法能够**学习随机策略**，这使得智能体在面对部分可观测或具有多样性需求的任务时表现更好。此外，在一定条件下，策略梯度的优化过程能够保证收敛到局部最优解，避免了Q函数逼近中的不稳定性问题。

然而，策略梯度方法也存在显著的挑战。其一，**梯度估计的方差较高**。由于期望回报的估计依赖采样，梯度信号往往噪声较大，从而导致更新不稳定。其二，**样本效率较低**。相比于利用价值函数进行自举（bootstrapping）的算法，策略梯度需要更多交互样本来获得可靠估计。其三，由于优化目标是非凸的，策略梯度方法通常只能保证**局部最优收敛**。此外，如何平衡探索与利用仍然是一个需要精心设计的问题。

综合来看，策略梯度方法通过直接优化策略函数，为强化学习提供了一种更具灵活性和理论优雅性的路径。

6.2 策略梯度定理

策略梯度定理是整个基于策略的强化学习方法的理论核心。它给出了期望累积回报关于策略参数的梯度形式，使得策略的更新可以直接通过梯度上升来优化目标函数，而无需显式地考虑状态分布随参数变化的复杂影响。

(1) 策略梯度定理的形式

策略梯度定理给出了目标函数关于策略参数 θ 的梯度表达式：

$$\nabla_{\theta} J(\theta) = \sum_s d^{\pi_{\theta}}(s) \sum_a \pi_{\theta}(a|s) \nabla_{\theta} \log \pi_{\theta}(a|s) Q^{\pi_{\theta}}(s, a).$$

其中， $d^{\pi_{\theta}}(s)$ 是在策略 π_{θ} 下状态 s 的平稳分布， $Q^{\pi_{\theta}}(s, a)$ 是在该策略下的动作价值函数。

进一步地，可以用期望符号简洁地表示为：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{s \sim d^{\pi_{\theta}}, a \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a|s) Q^{\pi_{\theta}}(s, a)].$$

策略梯度定理描述了期望回报的梯度取决于动作概率随参数变化的方向与程度。这一定理的意义在于，它将复杂的梯度计算问题转化为对状态和动作的采样期望，使得通过采样估计梯度成为可能。

(2) 策略梯度定理的证明

(a) 策略梯度方法的目标函数

Agent 与环境交互生成轨迹 $\tau = (s_0, a_0, s_1, a_1, \dots)$ ，其折扣回报为 $R(\tau) = \sum_{t=0}^{\infty} \gamma^t r_t$ 。策略的目标是最大化轨迹分布下回报的期望，因此目标函数可表示为：

$$J(\boldsymbol{\theta}) = \mathbb{E}_{\tau \sim p_{\boldsymbol{\theta}}}[R(\tau)] = \int p_{\boldsymbol{\theta}}(\tau) R(\tau) d\tau$$

其中，轨迹概率分布 $p_{\boldsymbol{\theta}}(\tau)$ 依赖于策略 $\pi_{\boldsymbol{\theta}}$ ：

$$p_{\boldsymbol{\theta}}(\tau) = \mu(s_0) \prod_{t=0}^{\infty} \pi_{\boldsymbol{\theta}}(a_t | s_t) \cdot P(s_{t+1} | s_t, a_t)$$

这里， $\mu(s_0)$ 是初始状态分布， $P(s_{t+1} | s_t, a_t)$ 是环境的状态转移概率，两者均与策略参数 $\boldsymbol{\theta}$ 无关。

(b) 目标函数的梯度

对目标函数关策略的参数求梯度：

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \nabla_{\boldsymbol{\theta}} \int p_{\boldsymbol{\theta}}(\tau) R(\tau) d\tau = \int \nabla_{\boldsymbol{\theta}} p_{\boldsymbol{\theta}}(\tau) R(\tau) d\tau$$

使用恒等式 $\nabla_{\boldsymbol{\theta}} p_{\boldsymbol{\theta}}(\tau) = p_{\boldsymbol{\theta}}(\tau) \nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}(\tau)$ ，代入得：

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \int p_{\boldsymbol{\theta}}(\tau) \nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}(\tau) R(\tau) d\tau = \mathbb{E}_{\tau \sim p_{\boldsymbol{\theta}}} [\nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}(\tau) R(\tau)]$$

(c) 轨迹对数概率的分解

根据轨迹概率分布的定义，其对数概率可表示为：

$$\log p_{\boldsymbol{\theta}}(\tau) = \log \mu(s_0) + \sum_{t=0}^{\infty} [\log \pi_{\boldsymbol{\theta}}(a_t | s_t) + \log P(s_{t+1} | s_t, a_t)]$$

由于 $\mu(s_0)$ 和 $P(s_{t+1} | s_t, a_t)$ 与 $\boldsymbol{\theta}$ 无关，其梯度为零，因此：

$$\nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}(\tau) = \sum_{t=0}^{\infty} \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a_t | s_t)$$

代入目标函数梯度：

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \mathbb{E}_{\tau \sim p_{\boldsymbol{\theta}}} \left[\left(\sum_{t=0}^{\infty} \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a_t | s_t) \right) R(\tau) \right]$$

(d) 引入动作价值函数

在上式所示的目标函数梯度中， $R(\tau)$ 是整个轨迹的回报。我们可以把 $R(\tau)$ 分解为过去回报和未来回报：

$$R(\tau) = \sum_{k=0}^{t-1} \gamma^k r_k + \gamma^t \sum_{k=t}^{\infty} \gamma^{k-t} r_k$$

其中，未来回报 $\sum_{k=t}^{\infty} \gamma^{k-t} r_k$ 是从时间 t 开始的未来折扣回报。

这里，需要我们特别注意的一个关键事实是：策略在时间 t 的动作 a_t 不会影响时间 t 之前的回报。这一事实意味着过去回报与当前动作无关。换言之，只有未来回报才会对目标函数的梯度产生贡献。

进一步地，我们还知道动作价值函数定义为状态 s_t 执行动作 a_t 后的期望折扣回报，即 $Q^{\pi_{\boldsymbol{\theta}}}(s_t, a_t) = \mathbb{E} [\sum_{k=0}^{\infty} \gamma^k r_{t+k} | s_t, a_t]$ 。也就是说，状态价值是未来回报的条件期望。

于是，目标函数的梯度可以表示为：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}} \left[\sum_{t=0}^{\infty} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot \gamma^t Q^{\pi_{\theta}}(s_t, a_t) \right]$$

(e) 策略梯度定理的最终形式

对于任意时间步 t ，状态 s_t 出现的概率由**折扣状态访问分布** $d^{\pi_{\theta}}(s)$ 描述：

$$d^{\pi_{\theta}}(s) = \sum_{t=0}^{\infty} \gamma^t P(s_t = s | \pi_{\theta})$$

其中， $P(s_t = s | \pi_{\theta})$ 表示在策略 π_{θ} 下，时间 t 时状态为 s 的概率。折扣因子 γ^t 确保了长期访问的状态具有较低的权重。

利用折扣状态访问分布，我们可以将目标函数梯度中，对轨迹的期望分解为对状态访问分布和动作选择的期望：

$$\begin{aligned} \nabla_{\theta} J(\theta) &= \sum_{t=0}^{\infty} \mathbb{E}_{s_t \sim P(s_t | \pi_{\theta}), a_t \sim \pi_{\theta}} [\gamma^t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) Q^{\pi_{\theta}}(s_t, a_t)] \\ &= \sum_s \left(\sum_{t=0}^{\infty} \gamma^t P(s_t = s | \pi_{\theta}) \right) \sum_a \pi_{\theta}(a | s) \nabla_{\theta} \log \pi_{\theta}(a | s) Q^{\pi_{\theta}}(s, a) \\ &= \sum_s d^{\pi_{\theta}}(s) \sum_a \pi_{\theta}(a | s) \nabla_{\theta} \log \pi_{\theta}(a | s) Q^{\pi_{\theta}}(s, a) \end{aligned}$$

策略梯度定理的最终形式为：

$$\boxed{\nabla_{\theta} J(\theta) = \sum_s d^{\pi_{\theta}}(s) \sum_a \pi_{\theta}(a | s) \nabla_{\theta} \log \pi_{\theta}(a | s) Q^{\pi_{\theta}}(s, a)}$$

也常表示为：

$$\boxed{\nabla_{\theta} J(\theta) = \mathbb{E}_{s \sim d^{\pi_{\theta}}, a \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a | s) Q^{\pi_{\theta}}(s, a)]}$$

(3) 策略梯度定理的作用

策略梯度定理的核心价值在于为强化学习中的策略优化问题提供了切实可行的解决方案。它将目标函数的梯度转化为一个简洁的期望形式：

$$\nabla_{\theta} J(\theta) = \mathbb{E} [\nabla_{\theta} \log \pi_{\theta}(a | s) Q^{\pi_{\theta}}(s, a)]$$

这一数学表达具有深刻的实际意义。它表明，**尽管目标函数本身涉及复杂的轨迹分布，但其梯度却可以通过采样得到的状态-动作对进行估计**。这使得策略优化不再依赖于对环境动态的完整知识，而是可以通过与环境的实际交互来学习。

基于这一定理，策略更新可以通过随机梯度上升实现：

$$\boxed{\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} \log \pi_{\theta}(a | s) \cdot \hat{Q}(s, a)}$$

其中，学习率 α 控制更新步长， $\hat{Q}(s, a)$ 是对动作价值的估计。这种更新方式使得算法能够在与环境交互的过程中逐步改进策略，体现了强化学习“在尝试中学习”的核心思想。

在实际实现中，我们可以构造如下**损失函数**：

$$L(\theta) = -\mathbb{E}_{s \sim d^{\pi_\theta}, a \sim \pi_\theta} [\log \pi_\theta(a|s) \cdot \hat{Q}^{\pi_\theta}(s, a)]$$

利用梯度下降方法最小化该损失函数，就等价于实现了策略梯度定理。

策略梯度定理不仅为经典的REINFORCE算法提供了理论依据，更重要的是，它构建了一个可扩展的算法框架。在此框架下，研究者可以通过改进价值估计方法来提升算法性能。

总而言之，策略梯度定理架起了理论分析与工程实践之间的桥梁。它将复杂的策略优化问题转化为可通过采样解决的随机优化问题，使得强化学习能够在机器人控制、游戏智能体、自动驾驶等复杂场景中得到有效应用。

6.3 REINFORCE算法

策略梯度定理表明，只要我们能够从策略 π_θ 与环境交互中采样状态—动作对 (s, a) ，并获得相应的动作价值估计 $Q^{\pi_\theta}(s, a)$ ，就可以通过梯度上升的方式优化策略参数。REINFORCE算法正是这种思想的最直接实现，因此被称为**蒙特卡罗策略梯度算法**（Monte Carlo Policy Gradient），它是策略梯度方法的最早形式之一，也是理解后续高级算法（如Actor-Critic等）的基础。

(1) REINFORCE算法的基本思想

REINFORCE的核心思想非常简单：通过完整采样若干条轨迹 $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T)$ ，计算每个时间步的**实际回报**，并以此近似 $Q^{\pi_\theta}(s_t, a_t)$ ，从而估计目标函数梯度。

根据策略梯度定理：

$$\nabla_\theta J(\theta) = \mathbb{E}_{s \sim d^{\pi_\theta}, a \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(a | s) Q^{\pi_\theta}(s, a)]$$

由于真实的 $Q^{\pi_\theta}(s_t, a_t)$ 难以精确计算，REINFORCE使用**蒙特卡罗回报** G_t 作为其无偏估计：

$$G_t = \sum_{k=t}^T \gamma^{k-t} r_k.$$

于是可以得到梯度的无偏估计：

$$\hat{g}_t = \nabla_\theta \log \pi_\theta(a_t | s_t) G_t$$

在实践中，REINFORCE通过对多个样本的平均来近似期望，从而实现梯度上升更新：

$$\theta \leftarrow \theta + \alpha \sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) G_t,$$

其中 $\alpha > 0$ 为学习率。

这种更新方式的直观含义非常清晰。如果某个动作 a_t 导致了较高的回报 G_t ，算法就会增加该动作在当前状态下被选择的概率；反之，如果回报较低，则会减小该动作被选择的概率。这种直接的“奖惩式”参数调整，体现了策略梯度的核心机制。

(2) 基线与方差优化

尽管REINFORCE在理论上无偏，但其梯度估计方差往往很高。这是因为每个样本的回报 G_t 不仅受当前动作影响，也受后续随机事件影响。为了降低这种方差，可以引入一个**基线函数**（baseline function）。

基线的思想是：在不改变梯度期望的情况下，从回报中减去一个不依赖于动作 a_t 的参考值 $b(s_t)$ ，从而减少样本波动。修正后的梯度估计为：

$$\hat{g}_t = \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (G_t - b(s_t)).$$

为了说明为什么引入基线不会引入偏差，考虑其期望：

$$\mathbb{E}_{a_t \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) b(s_t)] = b(s_t) \nabla_{\theta} \sum_{a_t} \pi_{\theta}(a_t | s_t) = b(s_t) \nabla_{\theta} 1 = 0.$$

因此，该修正保持了无偏性，但显著降低了梯度估计的方差。

基线有效性的另一种解释是，现实中的奖励往往为正数，这使得 G_t 也是一个正数，从而导致梯度估计总是倾向于增加动作概率。引入基线后，当 G_t 超过基线时，才会增加该动作的概率；否则会降低该动作的概率。从而，使得策略的更新更加平衡和稳定，这也是之所以梯度估计的方差会降低的一种直观理解。

常用的基线选择包括常数基线、以轨迹平均回报为基线，或者使用移动平均方法等。不过，在实际应用中，基线通常取为当前策略下的**状态价值函数** $V^{\pi_{\theta}}(s_t)$ ，即当前状态的平均期望回报。这种选择能够有效地反映“在该状态下的平均表现”，从而更准确地衡量动作的“相对好坏”：

$$\hat{g}_t = \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (G_t - V^{\pi_{\theta}}(s_t))$$

这种情况下，需要额外训练一个**价值网络**来估计 $V^{\pi_{\theta}}(s)$ ，通常通过最小化均方误差损失：

$$L(\mathbf{w}) = \mathbb{E}_{s_t \sim d^{\pi_{\theta}}} [(V(s_t; \mathbf{w}) - G_t)^2]$$

其中， $\mathbf{w}$ 为价值网络的参数。

(3) 优势函数

引入基线后，可以自然地将梯度更新项重写为优势函数（advantage function）的形式：

$$A^{\pi_{\theta}}(s_t, a_t) = Q^{\pi_{\theta}}(s_t, a_t) - V^{\pi_{\theta}}(s_t).$$

优势函数刻画了动作 a_t 相对于该状态下平均表现的优劣程度。若 $A^{\pi}(s_t, a_t) > 0$ ，则表示该动作比平均表现更好，策略更新应当增加其概率；反之则应减少其概率。

因此，基于优势函数的梯度表达式为：

$$\nabla_{\theta} J(\theta) = \mathbb{E} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) A^{\pi_{\theta}}(s_t, a_t)]$$

在REINFORCE中，若用蒙特卡罗回报 G_t 近似 $Q^{\pi_{\theta}}(s_t, a_t)$ ，并用独立估计器（如神经网络）近似 $V^{\pi_{\theta}}(s_t)$ ，就得到**带基线的REINFORCE算法**，或称为**带价值函数基线的策略梯度算法**。这也是后续Actor-Critic方法的切入点。

(4) REINFORCE算法流程

根据上述分析，REINFORCE的实现流程可概括如下：

- **轨迹采样**：使用当前策略 π_{θ} 与环境交互，生成一条或多条轨迹 $\tau = (s_0, a_0, r_0, \dots, s_T)$ 。

- **计算回报**：对每个时间步 t ，计算蒙特卡罗回报 $G_t = \sum_{k=t}^T \gamma^{k-t} r_k$ 。
- **基线调整**：若引入基线函数，则计算 $b(s_t)$ 或 $V(s_t)$ ，并令 $\delta_t = G_t - b(s_t)$ ；若未使用基线，则直接令 $\delta_t = G_t$ 。
- **参数更新**：根据策略梯度更新：

$$\theta \leftarrow \theta + \alpha \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \delta_t$$

该算法完全基于采样，不需要环境模型，因此属于**无模型（model-free）**的强化学习方法。由于其更新方向为回报加权的期望梯度，REINFORCE能够在连续或高维策略空间中直接优化参数。

- **例**：几种简单的基线实现
 - 17-reinforce_simple_baselines.py
- **例**：以状态价值函数为基线
 - 18-reinforce_statevalue_baseline.py

(5) 连续动作空间的REINFORCE

在离散动作空间中，策略 $\pi_{\theta}(a|s)$ 表示为在有限动作集合上的概率分布，智能体通过在该分布上采样动作并依据回报调整参数 θ 来优化策略。然而，在许多现实控制任务（如机械臂控制、倒立摆平衡或无人机姿态调节）中，动作变量是连续的，无法直接对所有动作取枚举式概率。此时，必须将策略表示为**连续概率密度函数**，并在其上定义可微分的采样机制，以便梯度传播与参数更新。

最常见的策略形式是**条件高斯分布（Gaussian policy）**。其基本思想是令动作在给定状态下服从一个参数化的正态分布：

$$\pi_{\theta}(a|s) = \mathcal{N}(a; \mu_{\theta}(s), \sigma_{\theta}^2(s)\mathbf{I})$$

其中， $\mu_{\theta}(s)$ 表示在状态 s 下动作的均值向量， $\sigma_{\theta}(s)$ 为标准差向量，两者均由策略网络输出。该分布保证了在每个状态下，智能体能够以一定随机性采样动作：

$$a_t \sim \pi_{\theta}(\cdot | s_t)$$

这种设计使策略在连续动作空间中仍可保持可微性，从而能够通过梯度上升来直接优化期望回报。

连续动作空间的 REINFORCE 方法实质上是将原本定义在离散动作集上的策略概率扩展为连续分布密度函数，并通过对分布参数的可微分建模，使得梯度能够直接作用于均值与方差，从而实现对连续控制策略的优化。其更新本质仍遵循“提高产生高回报动作的概率”这一原则。通过这种方式，REINFORCE不仅保留了策略梯度方法的理论优雅性，还能够自然适应连续控制任务中的随机性与光滑性要求。

在连续动作空间的 REINFORCE 算法中，虽然智能体的动作是通过从高斯分布等连续概率分布中采样得到的，但这些采样步骤本身并不需要可导。**采样的作用仅仅是生成智能体与环境交互的轨迹**，从而获得状态、动作与奖励序列，用于估计期望回报。而在参数更新阶段，**梯度的计算并不是通过采样路径传播的，而是直接作用在策略的概率密度函数上**。由于策略网络输出的对数概率 $\log \pi_{\theta}(a|s)$ 对参数 θ 是可导的，因此我们可以通过 $\nabla_{\theta} \log \pi_{\theta}(a|s)$ 来计算梯度，实现策略更新，而无需对采样过程本身求导。这种特点大大扩展了策略梯度方法乃至强化学习方法的应用范围，在任何存在不可微采样步骤的场景中，均可通过这种方式进行有效学习。

不过，REINFORCE在连续动作空间中的能力也受到一些限制。首先，REINFORCE依赖于完整的轨迹采样来估计回报，这在高维连续动作空间中可能导致样本效率低下。其次，梯度估计的方差问题在连续空间中尤为突出，因为动作的微小变化可能导致回报的剧烈波动，从而影响学习稳定性。为缓解这些问题，通常需要引入更复杂的基线函数（如状态价值函数）或采用更先进的策略优化方法，以提高样本利用率和梯度估计的可靠性。

- 例：连续动作空间的REINFORCE

- 19-reinforce_continuous_action.py

6.4 PPO算法

REINFORCE算法属于**同策略（on-policy）策略梯度方法**。这意味着每次策略更新都必须依赖于由当前策略 π_θ 采样得到的数据。然而，这种“边采样边学习”的机制使得样本利用率极低，因为每次更新后旧数据将不再可用，训练效率受到严重限制。

异策略（off-policy）策略梯度方法允许使用由旧策略（行为策略）产生的数据来优化新的策略（目标策略），显著提高样本利用率。不过，异策略学习的难点在于行为策略采样的分布与目标策略不同，导致梯度估计产生偏差。针对这一问题，重要的改进方法之一便是**PPO（Proximal Policy Optimization）算法**，通过优化目标中引入**重要性采样修正（importance sampling correction）**与**信赖域约束（trust region constraint）**，在保证策略改进稳定性的同时，大幅提高了训练效率，是目前最常用、最稳定的策略梯度方法之一。

（1）异策略策略梯度思想

假设当前要优化的目标策略为 $\pi_\theta(a|s)$ ，而采样数据是由行为策略 $\pi_{\theta_{\text{old}}}(a|s)$ 生成的。在这种情况下，直接使用行为策略的数据来估计目标策略的梯度会引入偏差。为了解决这个问题，可以使用**重要性采样（importance sampling）**技术对梯度进行修正。

重要性采样的核心思想是通过调整样本权重来补偿采样分布的差异。具体地，目标策略与行为策略的比值称为**重要性采样比（importance sampling ratio）**，定义为：

$$\rho_t = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

对于任意函数 $f(s, a)$ ，我们希望计算在目标策略 π_θ 下的期望：

$$\mathbb{E}_{a \sim \pi_\theta}[f(s, a)] = \sum_a \pi_\theta(a|s) f(s, a)$$

由于我们只能从行为策略 $\pi_{\theta_{\text{old}}}$ 采样，因此可以通过重要性采样将期望转换为在行为策略下的期望：

$$\mathbb{E}_{a \sim \pi_\theta}[f(s, a)] = \sum_a \pi_{\theta_{\text{old}}}(a|s) \frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)} f(s, a) = \mathbb{E}_{a \sim \pi_{\theta_{\text{old}}}}[\rho_t \cdot f(s, a)]$$

因此，当使用行为策略的采样数据时，在策略梯度估计中引入重要性采样比 ρ_t ，可以得到无偏的梯度估计：

$$\begin{aligned} \nabla_\theta J(\theta) &= \mathbb{E}_{s \sim d^{\pi_{\theta_{\text{old}}}}, a \sim \pi_{\theta_{\text{old}}}}[\nabla_\theta \log \pi_\theta(a|s) Q^{\pi_{\theta_{\text{old}}}}(s, a)] \\ &= \mathbb{E}_{s \sim d^{\pi_{\theta_{\text{old}}}}, a \sim \pi_{\theta_{\text{old}}}}\left[\frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)} \nabla_\theta \log \pi_\theta(a|s) Q^{\pi_{\theta_{\text{old}}}}(s, a)\right] \\ &= \mathbb{E}_{s \sim d^{\pi_{\theta_{\text{old}}}}, a \sim \pi_{\theta_{\text{old}}}}[\rho_t \nabla_\theta \log \pi_\theta(a|s) Q^{\pi_{\theta_{\text{old}}}}(s, a)] \end{aligned}$$

与REINFORCE类似，可以使优势函数 $A^{\pi_{\theta_{\text{old}}}}(s_t, a_t)$ 来近似 $Q^{\pi_{\theta_{\text{old}}}}(s_t, a_t)$ ，从而得到异策略的梯度估计：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{s \sim d^{\pi_{\theta_{\text{old}}}}, a \sim \pi_{\theta_{\text{old}}}} [\rho_t \cdot \nabla_{\theta} \log \pi_{\theta}(a|s) \cdot A^{\pi_{\theta_{\text{old}}}}(s, a)]$$

(2) 策略更新的稳定性问题

虽然利用重要性采样可以在理论上实现异策略学习，但在实践中它会带来严重的不稳定性。当目标策略与行为策略差异较大时，重要性采样比 ρ_t 的值可能远离1，导致梯度估计的方差剧增，从而使得策略更新过程不稳定，甚至发散。

为简单起见，我们将重要性采样表示为利用来自分布 q 的样本 x ，估计随机变量 $f(x)$ 关于分布 p 的期望：

$$\mathbb{E}_{x \sim p}[f(x)] = \int p(x) f(x) dx = \int \frac{p(x)}{q(x)} q(x) f(x) dx = \mathbb{E}_{x \sim q} \left[f(x) \frac{p(x)}{q(x)} \right]$$

重要性比率 $\frac{p(x)}{q(x)}$ 的方差直接影响估计的方差。在没有使用重要性采样时，方差可以表示为：

$$\text{Var}_{x \sim p}[f(x)] = \mathbb{E}_{x \sim p} [f(x)^2] - (\mathbb{E}_{x \sim p} [f(x)])^2$$

而当使用重要性采样时，方差变为：

$$\begin{aligned} \text{Var}_{x \sim q} \left[f(x) \frac{p(x)}{q(x)} \right] &= \mathbb{E}_{x \sim q} \left[\left(f(x) \frac{p(x)}{q(x)} \right)^2 \right] - \left(\mathbb{E}_{x \sim q} \left[f(x) \frac{p(x)}{q(x)} \right] \right)^2 \\ &= \mathbb{E}_{x \sim p} \left[\left(f(x) \frac{p(x)}{q(x)} \right)^2 \frac{q(x)}{p(x)} \right] - (\mathbb{E}_{x \sim p} [f(x)])^2 \\ &= \mathbb{E}_{x \sim p} \left[f(x)^2 \frac{p(x)}{q(x)} \right] - (\mathbb{E}_{x \sim p} [f(x)])^2 \end{aligned}$$

这就意味着，当 $p(x)$ 和 $q(x)$ 差异较大时，两者的方差会存在明显差异，导致估计不稳定。

为了解决这一问题，其中一种思路是在策略更新时引入**信赖域约束 (trust region constraint)**，限制目标策略和行为策略之间的差异，从而控制重要性采样比的范围。具体来说，在策略更新时，除了最大化目标函数外，还需要满足以下约束条件：

$$D_{\text{KL}}(\pi_{\theta_{\text{old}}}(\cdot|s) \parallel \pi_{\theta}(\cdot|s)) \leq \delta$$

其中， δ 是一个小的正数，表示允许的最大偏差。

通过控制目标策略分布和行为策略分布之间的KL散度，可以有效地限制重要性采样比 ρ_t 的范围，避免其过大或过小，从而提升梯度估计的稳定性。不过，这种方法通常需要复杂的二阶优化技术（如TRPO算法），计算开销较大。

(3) PPO算法的核心思想

PPO算法采用了一种更为简单且高效的方式来实现信赖域约束。其核心思想是通过**截断重要性采样比**来限制策略更新的幅度，从而避免过大的策略变动。具体来说，PPO引入了一个**剪切函数 (clipping function)**，将重要性采样比 ρ_t 限制在 $[1 - \epsilon, 1 + \epsilon]$ 范围内，其中 ϵ 是一个较小的正数，通常取值在0.1到0.3之间。这样，PPO需要最小化的目标函数可以表示为：

$$L^{\text{CLIP}}(\theta) = -\mathbb{E}_{s, a \sim \pi_{\theta_{\text{old}}}} [\min(\rho_t \cdot A^{\pi_{\theta_{\text{old}}}}(s, a), \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) \cdot A^{\pi_{\theta_{\text{old}}}}(s, a))]$$

其中， $\text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon)$ 表示将 ρ_t 限制在 $[1 - \epsilon, 1 + \epsilon]$ 范围内的操作。

该目标函数的含义是：对于每个状态-动作对 (s, a) ，我们首先计算两个值：

- **未剪切的目标**： $\rho_t \cdot A^{\pi_{\theta_{\text{old}}}}(s, a)$ ，表示按照重要性采样比调整后的优势函数。
- **剪切后的目标**： $\text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) \cdot A^{\pi_{\theta_{\text{old}}}}(s, a)$ ，表示将重要性采样比限制在一定范围内后的优势函数。

由于优势函数的取值可能为正或负，PPO通过取这两个值的最小值，确保了当 ρ_t 超出 $[1 - \epsilon, 1 + \epsilon]$ 范围时，策略更新不会过度依赖于该样本，从而避免了过大的策略变动。这种设计有效地控制了策略更新的幅度，提升了训练的稳定性。

在实际实现中，当我们对 $L^{\text{CLIP}}(\theta)$ 进行优化时，可以使用常规的梯度下降方法来更新策略参数 θ 。其中，第一项关于参数的梯度为：

$$\begin{aligned} \frac{\partial \rho_t \cdot A^{\pi_{\theta_{\text{old}}}}}{\partial \theta} &= \frac{\partial \rho_t}{\partial \theta} \cdot A^{\pi_{\theta_{\text{old}}}} \\ &= \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \cdot \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \cdot A^{\pi_{\theta_{\text{old}}}} \\ &= \rho_t \cdot \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \cdot A^{\pi_{\theta_{\text{old}}}} \end{aligned}$$

目标函数中第二项梯度的计算方式类似，这与策略梯度定理的形式一致。

(4) 优势函数的估计

在策略梯度方法中，优势函数 A_t 的准确估计对于减少方差、提升学习效率至关重要。由于直接使用回报或时序差分估计都会面临偏差与方差之间的矛盾，PPO引入了**广义优势估计 (Generalized Advantage Estimation, GAE)**，在这两者之间实现平衡。其核心思想是通过引入一个衰减因子 λ ，在单步时序差分与多步回报之间进行加权，从而在偏差和方差之间取得折中。

首先定义时序差分误差 (TD误差) 为：

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t),$$

其中， $V(s_t)$ 是价值函数对状态 s_t 的估计， $\gamma \in (0, 1]$ 是折扣因子。该项刻画了在时刻 t 的价值估计与实际回报之间的差异。

在GAE中，优势函数的估计被表示为：

$$A_t^{\text{GAE}} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l},$$

其中 $\lambda \in [0, 1]$ 是一个超参数。当 $\lambda = 1$ 时，GAE退化为类似蒙特卡罗方法的无偏估计，但方差较大；当 $\lambda = 0$ 时，GAE则退化为单步TD估计，方差较小但偏差更大。因此，通过调节 λ ，可以在偏差与方差之间灵活权衡，使得训练过程更加稳定。

GAE的优点在于，它不仅继承了时序差分方法的低方差特性，还保留了部分多步回报的信息，从而在理论和实践中均展现出较好的性能。这也是它在PPO算法中被广泛采用的原因。

(5) PPO算法流程

PPO (Proximal Policy Optimization) 可以看作是一种近似的异策略Actor-Critic方法。其核心目标是通过限制新旧策略之间的更新幅度来避免策略崩溃，同时保持较高的学习效率。在该框架下，策略网络与价值网络分别由参数 θ 与 ϕ 表示。算法的主要流程如下：

首先，初始化策略参数 θ_0 、价值函数参数 ϕ_0 、裁剪参数 ϵ 以及学习率 α 。在每一轮迭代中，算法依次执行如下步骤：

- **采样交互数据**：利用当前旧策略 $\pi_{\theta_{\text{old}}}$ 与环境交互，收集轨迹样本 (s_t, a_t, r_t, s_{t+1}) 。这些样本为后续的策略优化与价值函数更新提供数据基础。
- **计算回报与优势函数**：对于每个时间步 t ，计算折扣回报

$$R_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

并使用价值函数 $V_{\phi}(s_t)$ 以及GAE方法计算优势估计 A_t 。GAE在此处起到降低方差、提升稳定性的作用。

- **重要性采样比**：在异策略更新中，需要修正新旧策略分布差异，这一修正通过重要性采样比来完成：

$$r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}。$$

该比率衡量了新策略与旧策略在同一状态下对动作 a_t 的概率比值。

- **裁剪目标函数**：为了避免策略过度更新，PPO引入了裁剪目标函数：

$$L^{\text{CLIP}}(\theta) = -\mathbb{E}_t \left[\min \left(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t \right) \right],$$

其中 $\text{clip}(\cdot)$ 操作限制了 $r_t(\theta)$ 的更新幅度。通过取最小值，算法有效抑制了因重要性比过大而导致的不稳定更新。

- **价值函数损失**：除策略更新外，价值函数的训练同样重要。其损失函数通常采用均方误差形式：

$$L_V(\phi) = \mathbb{E}_t \left[\left(R_t - V_{\phi}(s_t) \right)^2 \right]。$$

该项用于提升价值函数对状态回报的拟合精度。

- **参数更新**：采用随机梯度方法分别更新策略参数与价值函数参数：

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} L^{\text{CLIP}}, \quad \phi \leftarrow \phi - \alpha \nabla_{\phi} L_V。$$

- **策略同步**：在完成更新后，将 $\theta_{\text{old}} \leftarrow \theta$ ，并开始下一轮采样与优化。

通过上述流程，PPO在保证稳定性的同时，能够不断改进策略表现。与早期的Trust Region Policy Optimization (TRPO) 相比，PPO在实现上更加简洁高效，却依然能够取得类似甚至更优的性能，因此成为近年来应用最广泛的深度强化学习算法之一。

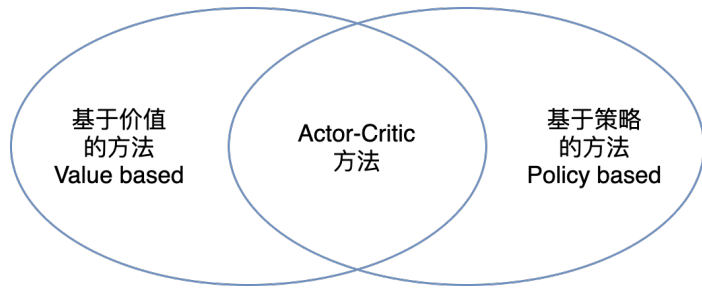
- 例：20-ppo.py

7. Actor-Critic 方法

强化学习的重要算法大致可以分为两类：基于价值的方法和基于策略的方法。基于价值的方法通过学习状态或状态—动作的价值函数来间接导出策略，例如Q-Learning或深度Q网络。它们的优点是训练相对稳定，更新规则简单，但在连续动作空间下求解最优动作困难，并且难以直接表示随机策略。

与之相对，基于策略的方法直接对策略函数进行参数化，并通过梯度优化性能指标来学习最优策略。它能够自然处理连续动作和随机策略，且策略更新方向明确。然而，这类方法通常面临梯度估计方差大、训练不稳定、收敛速度慢的问题。

这两类方法，前者高效而稳定，但缺乏可导性与策略表达能力；后者灵活且理论优美，却常因高方差而难以训练。自然的想法是融合二者的优点，即在保留价值估计的稳定性的同时，利用策略梯度的直接优化优势。这就是 Actor-Critic 方法的出发点。在 Actor-Critic 框架中，Actor 负责产生动作并优化策略，Critic 通过价值函数评价动作的好坏，为 Actor 提供低方差的学习信号。通过二者协同优化，Actor-Critic 既保留了策略方法的灵活性，又借助价值估计提高了训练稳定性，成为强化学习中重要的核心架构。



7.1 Actor-Critic 基本概念

(1) 基本思想

Actor-Critic 的核心思想可以概括为一句话：用一个模型（Critic）来评估另一个模型（Actor）的好坏。

在纯策略梯度方法中，估计式

$$\nabla_{\theta} J(\theta) = \mathbb{E} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t]$$

的方差往往较大。Actor-Critic 的关键改进在于：它将蒙特卡罗回报 G_t 替换为 Critic 给出的局部估计。这样得到的更新形式为：

$$\nabla_{\theta} J(\theta) \approx \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \hat{A}_{\omega}(s_t, a_t),$$

其中 $\hat{A}_{\omega}(s_t, a_t)$ 为优势函数，表示对当前动作相对优劣的估计。由于该估计通常由时序差分（TD）误差给出，因此能够在每一步更新中提供即时的梯度信号，大幅降低学习方差并提高样本利用率。

(2) Actor-Critic 架构核心组成

Actor-Critic 架构由 Actor 和 Critic 两个主要组成部分组成。其中 Actor 负责决策，即根据状态 s_t 产生动作 a_t ；Critic 则负责评价，即判断当前策略所采取动作的“好坏程度”。Actor 依据 Critic 的反馈不断调整自身参数，使策略逐渐趋向最优：

- **Actor**（执行者）：负责根据当前状态 s_t 选择动作 a_t ，其策略由参数 θ 参数化为 $\pi_{\theta}(a|s_t)$ 。Actor 的目标是通过学习优化策略，以最大化长期累积奖励。

- **Critic**（评价者）：负责评估Actor所采取动作的“好坏”，通常通过价值函数（如状态价值函数 $V_{\omega}(s)$ 或动作价值函数 $Q_{\omega}(s, a)$ ）实现，其中 ω 是Critic的参数。Critic使用时序差分（TD）误差来量化当前价值估计的偏差，并提供反馈给Actor。

在Actor-Critic架构中，另一个重要的概念是**优势函数（Advantage Function）**，它是由 Critic 计算产生的、用于指导 Actor 策略更新的核心评价信号。定义为动作价值函数与状态价值函数之差：

$$A^{\pi}(s_t, a_t) = Q^{\pi}(s_t, a_t) - V^{\pi}(s_t)$$

其优核心作用是定量评估特定动作的相对好坏：在给定状态 s_t 下，执行动作 a_t 相较于遵循当前策略 π 的平均表现而言，究竟有多好或多差。一个正的优势值鼓励Actor在未来更频繁地选择该动作，而负值则起到抑制作用。同时，通过减去状态价值 $V^{\pi}(s_t)$ 这一基线，它有效降低了梯度估计的方差，从而极大地稳定了训练过程。

在实践中，真实的优势函数是未知的，需要由Critic进行估计。其中，**时序差分误差** 是最常见且直接的一种估计形式：

$$\hat{A}(s_t, a_t) \approx \delta_t = r_t + \gamma V^{\pi}(s_{t+1}) - V^{\pi}(s_t)$$

这种方式之所以有效，是因为TD误差 δ_t 本质上是真实动作价值 $Q^{\pi}(s_t, a_t)$ 的一个单步无偏估计与当前状态价值 $V^{\pi}(s_t)$ 的差值，恰好吻合了优势函数的定义。除此之外，广义优势估计（GAE）等方法也能提供更平滑、偏差更低的优势估计。

(3) 与 REINFORCE+Baseline 的比较

值得注意的是，在策略梯度方法中，REINFORCE 算法往往通过引入 baseline 来降低梯度估计的方差。当 baseline 选取为由参数化模型 $V_{\omega}(s)$ 表示的状态价值函数时，这一结构在形式上已经与 Actor-Critic 相似：Actor 负责更新策略参数 θ ，而 Critic 则通过提供价值函数作为 baseline 来辅助策略优化。然而，REINFORCE+baseline 与真正的 Actor-Critic 仍存在本质区别。前者的价值函数仅在回合结束后利用完整回报 G_t 进行更新，因此 Critic 的作用主要体现在方差的减小；而后者的 Critic 则通过时序差分（TD）方法进行增量学习，直接提供近似优势函数，从而使策略能够在每个时间步得到更新信号。换言之，可以将 REINFORCE+baseline 视为一种特殊的 Actor-Critic 变体，其中 Critic 采用蒙特卡罗估计，而广义的 Actor-Critic 则通过 TD 误差在效率与方差控制之间实现了更好的平衡。

特性	REINFORCE + Baseline	Actor-Critic
Critic 的角色	作为仅用于降低方差的Baseline	作为策略更新的核心指导，提供优势函数估计
Critic 的学习信号	蒙特卡洛法，使用从当前状态到回合结束的完整回报 G_t 。	时序差分法，使用一步或多步即时奖励和下一状态价值的 bootstrap估计
策略更新时机	回合制，必须等待整个回合结束后才能进行更新。	在线/增量式，可以在每个时间步或每几个时间步后进行更新
效率与方差	相比无baseline方差较低，效率低（需要完整回合）	在效率和方差之间取得了更好的平衡，TD方法方差低于MC，且在线学习效率更高

7.2 时序差分 Actor-Critic

(1) Critic 的时序差分更新

在时序差分（TD）Actor-Critic 方法中，考虑采用状态价值函数 $V_{\omega}(s)$ 作为 Critic。基于 1-步 TD 的目标，定义 TD 目标和 TD 误差为：

$$\begin{aligned}\text{TD 目标: } y_t^{\text{TD}} &= r_t + \gamma V_{\omega}(s_{t+1}) \\ \text{TD 误差: } \delta_t &= y_t^{\text{TD}} - V_{\omega}(s_t) = r_t + \gamma V_{\omega}(s_{t+1}) - V_{\omega}(s_t)\end{aligned}$$

当 V_{ω} 恰好等于真实的 V^{π} 时，有：

$$\begin{aligned}\mathbb{E}[\delta_t \mid s_t, a_t] &= \mathbb{E}[r_t + \gamma V^{\pi}(s_{t+1}) - V^{\pi}(s_t) \mid s_t, a_t] \\ &= Q^{\pi}(s_t, a_t) - V^{\pi}(s_t) = A^{\pi}(s_t, a_t)\end{aligned}$$

因此， δ_t 在精确Critic下是优势函数的无偏估计；在近似Critic下， δ_t 通常仍然是对优势的有用低方差估计。正因为如此，我们可以使用 δ_t 来替代 A^{π} 作为 Actor 的学习信号。

当采用最常见的平方 TD 误差最小化时，Critic 的损失函数为：

$$L(\omega) = \frac{1}{2} \mathbb{E}[(y_t^{\text{TD}} - V_{\omega}(s_t))^2] = \frac{1}{2} \mathbb{E}[\delta_t^2].$$

对 ω 做随机梯度下降得到更新：

$$\omega \leftarrow \omega - \alpha_c \nabla_{\omega} L(\omega)$$

其中 $\alpha_c > 0$ 为 Critic 的学习率。

更准确地说，这里使用了半梯度（semi-gradient）方法以保持算法稳定。因为在计算 $\nabla_{\omega} y_t^{\text{TD}} = \nabla_{\omega}(r_t + \gamma V_{\omega}(s_{t+1}))$ 时我们忽略了目标对 ω 的依赖。

Critic 也可以使用动作价值函数 $Q_{\omega}(s, a)$ 。这种情况下采用类似的 TD 目标（例如 SARSA 或 Q-learning 风格的目标），并相应地更新 ω 。

(2) Actor 的策略梯度更新

在时序差分的情况下，使用 TD 误差 δ_t 作为优势函数的估计，Actor 的策略梯度更新为：

$$\theta \leftarrow \theta + \alpha_a \nabla_{\theta} \log \pi_{\theta}(a_t \mid s_t) \cdot \delta_t$$

其中 $\alpha_a > 0$ 为 Actor 的学习率。直观上，若 $\delta_t > 0$ （所采取动作比 Critic 预期更好），则增加该动作在状态 s_t 下的概率；反之则减少。

在 Actor-Critic 框架中，Actor 的更新满足策略梯度定理。具体来说，将策略梯度定理 $\nabla_{\theta} J(\theta) = \mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(a_t \mid s_t) Q^{\pi}(s_t, a_t)]$ 中的 $Q^{\pi}(s_t, a_t)$ 分解为 $V^{\pi}(s_t) + A^{\pi}(s_t, a_t)$ ，并利用基线 $V^{\pi}(s_t)$ 不改变期望的特点，得到：

$$\nabla_{\theta} J(\theta) = \mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(a_t \mid s_t) A^{\pi}(s_t, a_t)].$$

当 V_{ω} 精确时， $\mathbb{E}[\delta_t \mid s_t, a_t] = A^{\pi}(s_t, a_t)$ ，因此替换为 δ_t 是合适的、在期望意义上无偏的估计；当 V_{ω} 为近似时， δ_t 可能引入偏差，但通常能显著降低方差，从而使学习更稳定。

(3) Actor-Critic 算法流程

Actor-Critic 的主要循环由如下步骤组成（逐步在线更新形式）：

1. 在当前状态 s_t 下，根据策略 π_{θ} 采样动作 a_t ；

2. 与环境交互，接收奖励 r_t 和下一个状态 s_{t+1} ;
3. 计算 TD 目标 $y_t^{\text{TD}} = r_t + \gamma V_{\omega}(s_{t+1})$ 和 TD 误差 $\delta_t = y_t^{\text{TD}} - V_{\omega}(s_t)$;
4. 使用 δ_t 更新 Critic:

$$\omega \leftarrow \omega + \alpha_c \delta_t \nabla_{\omega} V_{\omega}(s_t)$$

5. 使用相同的 δ_t 更新 Actor:

$$\theta \leftarrow \theta + \alpha_a \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot \delta_t$$

6. $t \leftarrow t + 1$ ，重复以上步骤直至终止。

该流程可以直接在在线或仿真环境中实现，是诸多 Actor-Critic 变体（如 A2C、A3C、DDPG 的一个核心子过程）的基础。

- 例：21-actor-critic.py

7.3 A3C/A2C 算法

A3C（Asynchronous Advantage Actor-Critic）和 A2C（Advantage Actor-Critic 的同步化实现）是基于 Actor-Critic 思想发展而来的两个实用且高效的深度强化学习算法。它们的设计目标是提高训练效率、降低样本相关性并稳定策略学习；核心技术包括优势估计、并行采样（A3C 的异步多线程或 A2C 的同步多进程）以及对策略的正则化（熵项）。

(1) 优势函数

我们知道，策略梯度的优势形式为：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) A^{\pi}(s_t, a_t)],$$

其中优势函数定义为 $A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$ 。在 Actor-Critic 中，Critic 提供对 V^{π} （或 Q^{π} ）的估计，从而得到优势估计 $\hat{A}$ 用于 Actor 更新。

A3C/A2C 的关键是采用**多步回报**或**广义优势估计（GAE）**来构造低方差而偏差可控的 $\hat{A}$ 。

设采用 n -步回报（或截断的多步回报），定义从时刻 t 的 n -步回报（对固定 n ）为

$$G_t^{(n)} = r_t + \gamma r_{t+1} + \cdots + \gamma^{n-1} r_{t+n-1} + \gamma^n V_{\omega}(s_{t+n}),$$

则对应的 n -步优势估计为

$$\hat{A}_t^{(n)} = G_t^{(n)} - V_{\omega}(s_t)$$

在实践中，A3C/A2C 常用一个固定的步长 n （例如 5、20 的经验值），在每个工作线程/进程上收集若干步后再进行一次梯度更新。采用多步回报能在单步 TD 与整条轨迹的蒙特卡罗之间取得平衡，既减少方差又降低偏差。

若采用广义优势估计（GAE），则通过对不同 n -步优势的加权平均得到：

$$\hat{A}_t^{\text{GAE}(\lambda)} = \sum_{n=1}^{\infty} (\gamma \lambda)^{n-1} \hat{A}_t^{(n)} = \sum_{n=1}^{\infty} (\gamma \lambda)^{n-1} (G_t^{(n)} - V_{\omega}(s_t))$$

其中 $\lambda \in [0, 1]$ 为平衡偏差与方差的超参数。 $\lambda = 0$ 时退化为单步 TD， $\lambda = 1$ 时退化为蒙特卡罗估计。当 $\lambda = 1$ 时，GAE 退化为基于整条轨迹的蒙特卡罗回报（低偏差、高方差）。在实践中 λ 常取 0.9 左右以取得良好折中。GAE 通过指数加权平均多步优势，进一步降低了估计的方差，同时保留了多步信息，因而在 A3C/A2C 中被广泛采用。

(2) 熵正则化与复合损失

为了避免策略过早塌缩为确定性分布，A3C 在策略的目标中加入熵正则项以鼓励探索。定义策略熵为

$$\mathcal{H}(\pi_{\theta}(\cdot | s)) = - \sum_a \pi_{\theta}(a | s) \log \pi_{\theta}(a | s)$$

（离散动作）或相应的连续策略熵表达。A3C 的单步（或批量）目标通常以如下复合损失表示（训练时我们最小化负目标）：

Actor 的损失（取负以便最小化）

$$L_{\text{actor}}(\theta) = -\mathbb{E} \left[\log \pi_{\theta}(a_t | s_t) \hat{A}_t \right] - \beta_{\text{ent}} \mathbb{E} [\mathcal{H}(\pi_{\theta}(\cdot | s_t))]$$

Critic 的损失

$$L_{\text{critic}}(\omega) = \frac{1}{2} \mathbb{E} [(G_t^{(n)} - V_{\omega}(s_t))^2]$$

（离散动作）或相应的连续策略熵表达。A3C 的单步（或批量）目标通常以如下复合损失表示（训练时我们最小化负目标）：

$$L(\theta, \omega) = L_{\text{actor}}(\theta) + c_v L_{\text{critic}}(\omega)$$

其中 $c_v > 0$ 为价值损失的权重系数， $\beta_{\text{ent}} > 0$ 为熵正则化系数，用以平衡策略改进与探索。训练时对 θ 和 ω 分别或联合使用随机梯度优化（如 RMSProp、Adam 等）更新参数；在 A3C 的原始实现中，使用 RMSProp 并将各个线程的梯度异步应用到共享参数上。

(3) 异步并行与A3C

异步并行是 A3C 的核心工程创新之一。单个 agent 连续与环境交互时，采样路径之间强相关，这会导致训练样本的自相关性增大，从而影响优化的稳定性。A3C 的策略是启动多个并行的工作线程（或进程），每个线程在自己的环境副本中以当前共享策略进行交互并收集经验，然后将其梯度异步地发送到共享的全局参数（Actor 与 Critic 的参数均为共享）。这种设计带来几个好处：

- 并行工作使得采样更高效，能够充分利用多核 CPU（在当时比 GPU 更普遍）。
- 不同工作线程由于环境状态不同、随机性不同，生成了更多样化的样本，降低了样本相关性。
- 异步更新在一定程度上充当了随机扰动，帮助避免局部极值并加速收敛。

A3C 的异步更新流程大致为：每个工作线程拷贝全局参数的当前值到本地，执行若干步环境交互（或直到终止），计算本地的策略梯度与价值梯度，然后将这些梯度异步地应用到全局参数，并把更新后的全局参数重新拷贝到本地，重复上述循环。A3C 的这种去中心化但共享参数的策略，既保留了 on-policy 的优点（每次更新使用的是当前策略下的样本），又提高了样本效率与并行性。

(4) 同步化与A2C

A2C 是对 A3C 的一个同步化实现，其核心思路是把多个并行工作者产生的经验在批次上同步计算梯度，然后在主进程上一次性更新全局参数。A2C 保留了并行采样带来的样本多样性，但使得更新更稳定（因为在每次更新时使用了来自多个环境的平均梯度）。具体流程是：使用 M 个工作进程并行地从每个进程收集 n 步的经验，合并成一个批次后计算策略与价值的梯度平均值，然后用该平均梯度更新全局参数。A2C 在工程实现上更便于在单台机器的多核环境或分布式环境中实现，也更容易与现代深度学习框架的同步优化器配合。

(5) A2C 算法流程

下面给出 A2C 的伪代码。

初始化全局参数 θ （策略）、 ω （价值）

并行启动 M 个工作进程

每个工作进程重复：

同步本地参数 $\theta' \leftarrow \theta, \omega' \leftarrow \omega$

$t_{\text{start}} \leftarrow t$

收集 $t = t_{\text{start}} \dots t_{\text{start}} + k - 1$ 的经验：

在 s_t 下根据 $\pi_{\{\theta'\}}$ 采样 a_t ，执行获得 r_t, s_{t+1}

如果 episode 终止则 break

计算批次中的回报 $G_t^{\{(k)\}}$ （或使用 GAE 计算优势）

计算局部梯度：

$g_{\text{actor}} \leftarrow \sum_t \nabla_{\{\theta'\}} \log \pi_{\{\theta'\}}(a_t | s_t) * \hat{A}_t - \beta_{\text{ent}} * \nabla_{\{\theta'\}} H(\pi_{\{\theta'\}}(. | s_t))$

$g_{\text{critic}} \leftarrow \sum_t (G_t^{\{(k)\}} - V_{\{\omega'\}}(s_t)) * \nabla_{\{\omega'\}} V_{\{\omega'\}}(s_t)$

将 $g_{\text{actor}}, g_{\text{critic}}$ 发送至主进程（或直接更新全局参数）

主进程接收并应用平均/累加的梯度以更新 θ, ω

• 例：22-a2c.py

A3C 与 A2C 的差别是：每个工作线程直接将本地计算的梯度异步地应用到全局参数，而不等待其他线程，这样可以更频繁地更新全局参数并充分利用 CPU 并行性。

在工程实现中，A3C/A2C 的性能受多种细节影响：

- 对回报或优势进行**归一化**可以显著提高训练稳定性。常见做法是对批量内的优势做均值方差归一化 $\hat{A}_t \leftarrow (\hat{A}_t - \mu) / (\sigma + \epsilon)$ ；
- 对策略梯度**裁剪梯度范数**（gradient clipping）能够防止梯度爆炸并提高稳定性。再次，批次内对价值损失与策略损失使用不同权重（ c_v ）可以调节 Critic 与 Actor 的学习速度；
- 在许多实现中价值权重为 0.5 是常见选择。熵系数 β_{ent} 的选择影响探索程度，通常在训练初期取较大值逐渐衰减；
- A3C 的异步更新需要注意线程安全与参数共享。如果直接在共享内存中异步写参数，需用原子操作或框架支持的异步优化器；在现代实践中，A2C 的同步批量更新与分布式同步优化器更常用，因为它更易于与 GPU 加速和现代深度学习库集成；
- A3C/A2C 为同策略算法，因而不能直接使用经验回放。

7.4 DDPG

DDPG (Deep Deterministic Policy Gradient) 是一种面向连续动作空间的、基于 Actor-Critic 的异策略强化学习算法。它将确定性策略梯度理论 (Deterministic Policy Gradient, DPG) 与深度函数逼近、经验回放和目标网络等技巧结合起来, 从而在高维连续控制问题上取得了良好的实践效果。DDPG 的设计动机是: 许多实际控制任务的动作是连续的, 在这种情况下使用确定性策略 (即在给定状态下直接输出一个确定性动作向量) 在样本效率上往往优于等维的随机策略; 同时, 把经验缓存起来并反复利用可以显著提高样本效率。

(1) 确定性策略与目标函数

在基于策略的方法中, 我们通常采用随机策略 $\pi_{\theta}(a|s)$, 通过在给定状态下采样动作实现探索。然而, 当动作空间维度较高时, 随机策略的采样效率会显著降低。**确定性策略 (Deterministic Policy)** 提供了另一种思路: 在每个状态 s 下直接输出一个具体动作

$$a = \mu_{\theta}(s),$$

其中 μ_{θ} 是由参数 θ 控制的确定性映射函数 (即 Actor 网络)。

与随机策略一样, 确定性策略的目标仍是最大化长期累积回报的期望。我们定义性能目标为

$$J(\theta) = \mathbb{E}_{s \sim \rho^{\mu}}[R(s)],$$

其中 $\rho^{\mu}(s)$ 表示在策略 μ_{θ} 下的状态分布 (可理解为折扣加权的平稳分布); $R(s)$ 表示从状态 s 出发的期望累积回报, 即 $R(s) = \mathbb{E}[G_t | S_t = s] = V^{\mu}(s)$ 。

确定性策略梯度定理 (Deterministic Policy Gradient Theorem) 给出了性能目标关于参数的梯度形式:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{s \sim \rho^{\mu}} [\nabla_{\theta} \mu_{\theta}(s) \nabla_a Q^{\mu}(s, a) \Big|_{a=\mu_{\theta}(s)}]$$

这一结果揭示了策略改进的核心机制: 策略参数的变化通过动作 $\mu_{\theta}(s)$ 影响动作价值 $Q^{\mu}(s, a)$, 从而改变总体性能。换言之, 梯度可视为两个因素的乘积——“策略输出对参数的敏感度” ($\nabla_{\theta} \mu$) 与“动作对价值的敏感度” ($\nabla_a Q$)。在状态分布上对这一乘积取期望即可得到整体的性能提升方向。

确定性策略梯度定理可以理解为, 将随机策略 $\pi_{\theta}(a|s)$ 看作高斯分布族, 并令方差趋近于零, 得到确定性极限。

与随机策略梯度 (Stochastic Policy Gradient) 相比, 确定性形式不再对动作空间进行采样, 而是通过确定性映射直接计算梯度。这种方式在高维连续动作空间中具有显著的采样效率优势, 因此成为深度确定性策略梯度 (Deep Deterministic Policy Gradient, DDPG) 等算法的理论基础。

(2) Critic 与 Actor 的参数化目标

在实际算法中, 我们用参数化的动作价值函数 $Q_{\omega}(s, a)$ 近似真实的 $Q^{\mu}(s, a)$ 。Critic 的训练目标是使 Q_{ω} 满足贝尔曼方程的近似, 从而对动作的价值提供有效的梯度信号; Actor 则使用 Critic 的梯度信息按确定性策略梯度式进行更新。

Critic 的常用离线/小批量训练目标基于均方 TD 误差。引入目标网络 (后述) $\mu_{\theta'}$ 与 $Q_{\omega'}$ 来构造稳定的 TD 目标, 定义标量目标

$$y_t = r_t + \gamma Q_{\omega'}(s_{t+1}, \mu_{\theta'}(s_{t+1}))$$

Critic 的损失为小批量样本期望下的均方误差:

$$L(\omega) = \frac{1}{N} \sum_{i=1}^N \frac{1}{2} (y_i - Q_{\omega}(s_i, a_i))^2$$

对 ω 做梯度下降得到半梯度更新：

$$\omega \leftarrow \omega - \alpha_c \frac{1}{N} \sum_{i=1}^N (Q_{\omega}(s_i, a_i) - y_i) \nabla_{\omega} Q_{\omega}(s_i, a_i)$$

Actor 的目标是最大化 Critic 给出的动作价值。使用确定性策略梯度近似，Actor 的参数更新方向为

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \mu_{\theta}(s_i) \nabla_a Q_{\omega}(s_i, a) \Big|_{a=\mu_{\theta}(s_i)}$$

实际实现中常采用梯度上升（或等价地最小化负值）：

$$\theta \leftarrow \theta + \alpha_a \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \mu_{\theta}(s_i) \nabla_a Q_{\omega}(s_i, a) \Big|_{a=\mu_{\theta}(s_i)}$$

在自动微分框架下，等价的实现方式是对下式做梯度下降：

$$L_{\text{actor}}(\theta) = -\frac{1}{N} \sum_{i=1}^N Q_{\omega}(s_i, \mu_{\theta}(s_i))$$

并通过反向传播自动得到上面的链式梯度。

(3) 经验回放与目标网络

DDPG 在实践中借鉴了 DQN 的两大工程技巧：经验回放（replay buffer）与目标网络（target network）。这两者在异策略的深度学习场景下对稳定性至关重要。

经验回放是一个环形缓冲区，用于存储智能体与环境交互得到的四元组 (s_t, a_t, r_t, s_{t+1}) 。训练时从缓冲区按小批量随机采样，以打破样本间的时序相关性并提高样本利用率。经验回放使得 Critic 与 Actor 的更新变为离策略的、基于小批量的最优化问题，从而适配现代的随机梯度优化器。

目标网络是对 Actor/ Critic 的慢速副本，用以计算 TD 目标 y_t 。记主网络参数为 (θ, ω) ，对应的目标网络参数为 (θ', ω') 。在每次训练步骤之后，目标网络以“软更新”（Polyak averaging）的方式逼近主网络：

$$\theta' \leftarrow \tau \theta + (1 - \tau) \theta', \quad \omega' \leftarrow \tau \omega + (1 - \tau) \omega',$$

其中 $(\tau \ll 1)$ （例如 (0.001)）保证目标网络参数缓慢跟随主网络，从而使 TD 目标的变化更加平滑，减少训练过程中的震荡。

经验回放与目标网络结合，使得 Critic 的目标 y_t 更稳定、样本更独立，有助于收敛性与数值稳定性。

(4) 基于噪声过程的探索策略

DDPG 是离策略算法，因此探索是通过在执行阶段对确定性策略添加噪声实现的。常见的噪声选择包括独立同分布的高斯噪声与具有时间相关性的 Ornstein–Uhlenbeck（OU）噪声。OU 噪声在物理控制任务中常被采用，因为它生成平滑、具有惯性的噪声轨迹；而简单的高斯噪声在很多现代实现中也能取得同样甚至更好的效果。

噪声强度随训练进程逐渐衰减通常是有益的：训练早期用较大噪声促进探索，训练后期减小噪声以便策略收敛并进行精细控制。

(5) DDPG 算法结构

DDPG 结合了确定性策略梯度理论与深度神经网络函数逼近，采用 Actor-Critic 架构来分别表示策略和价值函数。其核心组件包括：

- **Actor 网络** $\mu_{\theta}(s)$ ：一个参数化的神经网络，输入状态 s ，输出确定性动作 a 。
- **Critic 网络** $Q_{\omega}(s, a)$ ：一个参数化的神经网络，输入状态-动作对 (s, a) ，输出对应的动作价值估计。
- **目标网络**：为了提高训练稳定性，DDPG 引入了目标网络 $\mu_{\theta'}$ 和 $Q_{\omega'}$ ，它们是 Actor 和 Critic 网络的延迟副本，通过软更新（soft update）逐步跟踪主网络的参数。
- **经验回放缓冲区**：DDPG 使用经验回放机制存储过去的交互数据 (s_t, a_t, r_t, s_{t+1}) ，并从中随机采样小批量数据进行训练，以打破数据间的相关性，提高样本利用率。

DDPG 的训练过程包括以下几个关键步骤：

1. **数据采集**：在环境中使用当前 Actor 策略 μ_{θ} 选择动作，并添加噪声（如 Ornstein-Uhlenbeck 噪声）以促进探索，执行动作并存储经验到回放缓冲区。
2. **Critic 更新**：从回放缓冲区随机采样小批量经验，计算目标值

$$y_t = r_t + \gamma Q_{\omega'}(s_{t+1}, \mu_{\theta'}(s_{t+1}))$$

并通过最小化均方误差损失

$$L(\omega) = \mathbb{E}_{(s_t, a_t, r_t, s_{t+1})} [(Q_{\omega}(s_t, a_t) - y_t)^2]$$

来更新 Critic 参数 ω 。

3. **Actor 更新**：利用 Critic 的梯度信息更新 Actor 参数 θ ，具体为

$$\theta \leftarrow \theta + \alpha_a \mathbb{E}_{s_t} [\nabla_{\theta} \mu_{\theta}(s_t) \nabla_a Q_{\omega}(s_t, a) \Big|_{a=\mu_{\theta}(s_t)}]$$

4. **目标网络更新**：通过软更新策略逐步调整目标网络参数

$$\theta' \leftarrow \tau \theta + (1 - \tau) \theta', \quad \omega' \leftarrow \tau \omega + (1 - \tau) \omega'$$

其中 $\tau \ll 1$ 是软更新系数。

(6) DDPG 算法流程

DDPG 的训练流程如下：

初始化主网络参数 θ (Actor), ω (Critic)

初始化目标网络参数 $\theta' \leftarrow \theta$, $\omega' \leftarrow \omega$

初始化空的经验回放缓冲区 R

初始化探索噪声过程 N (例如 OU 或高斯)

for episode = 1 to M do

 初始化环境状态 s_0

```

for t = 0 to T-1 do
    以确定性策略选择动作并加噪声:  $a_t = \mu_{\theta}(s_t) + N_t$ 
    与环境交互, 得到  $r_t, s_{t+1}$ 
    存储转移  $(s_t, a_t, r_t, s_{t+1})$  到 R
    从 R 中均匀随机采样小批量  $\{(s_i, a_i, r_i, s'_i)\}_{i=1}^N$ 
    计算目标  $y_i = r_i + \gamma Q_{\omega'}(s'_i, \mu_{\theta'}(s'_i))$ 
    更新 Critic:  $\omega \leftarrow \omega - \alpha_c * \nabla_{\omega} (1/N) \sum_i 1/2 (Q_{\omega}(s_i, a_i) - y_i)^2$ 
    更新 Actor:  $\theta \leftarrow \theta + \alpha_a * (1/N) \sum_i \nabla_{\theta} \mu_{\theta}(s_i) \nabla_a Q_{\omega}(s_i, a)|_{a=\mu_{\theta}(s_i)}$ 
    软更新目标网络:
         $\theta' \leftarrow \tau \theta + (1-\tau) \theta'$ 
         $\omega' \leftarrow \tau \omega + (1-\tau) \omega'$ 
     $s_t \leftarrow s_{t+1}$ 
    如果 episode 终止则 break

```

• 例: 23-ddpg.py

在评估时, 一般使用纯确定性策略 $a = \mu_{\theta}(s)$ (不加探索噪声) 以获得真实的策略性能。

在实际实现中, 可以使用如下方法来改善 DDPG 的稳定性与性能:

- 使用双 Critic (两个独立的 Q 网络) 并在计算目标时取较小者 (clipped double Q) 可以减少过估计偏差;
- 延迟更新 Actor 与目标网络 (即每经过若干次 Critic 更新才更新一次 Actor) 可以让 Critic 的估计更成熟后再对 Actor 进行优化, 避免误导性的梯度;
- 在计算目标时对目标动作加入微小的随机噪声并裁剪 (target policy smoothing) 可以减弱策略在训练过程中的过拟合问题。

7.5 Actor-Critic 方法的发展

Actor-Critic 家族自提出以来经历了两条互补的发展主线: 一条是**算法层面的理论与样本效率改进**, 另一条是**工程化与可扩展性的提升**。在算法层面, 最显著的进展之一是将最大熵 (maximum-entropy) 原则与离线/异策略 Actor-Critic 结合, 产生了 Soft Actor-Critic (SAC) 等算法; SAC 通过在目标中同时最大化期望回报与策略熵, 既提高了探索性又显著改进了训练稳定性与样本效率, 成为近年连续控制任务中的基准方法之一。其思想把 Actor-Critic 的目标从单纯的期望回报扩展为“回报 + 熵”, 从而在优化目标与探索策略之间引入了显式的权衡。

与 SAC 并列推动离策略 Actor-Critic 实用化的还有针对 DDPG 脆弱性的工程化修正, 例如 TD3 (Twin Delayed DDPG)。TD3 的三条核心改进 (双重 Critic 减少过估计、目标平滑以缓解误差放大、以及延迟更新 Actor) 直接解决了 DDPG 在函数逼近下常见的估计偏差与不稳定问题, 这些思想后来也被广泛借鉴到其它离策略 Actor-Critic 算法中。

另一方面, 随着计算资源与任务规模的增长, Actor-Critic 的**分布式与并行化**研究成为必然。Early 的 A3C 表明多线程并行能提高样本多样性并加速收敛, 而后来的 IMPALA 引入了 actor-learner 拆分与 V-trace 校正以实现极大规模、多任务训练的稳定性; 更近的工程化方案如 SEED-RL 则把推理集中化并用更加高效的通信与加速器利用策略来提高训练吞吐量与成本效率。这些工作把 Actor-Critic 框架从单机原型推向了大规模生产级训练的路径。

在 **离线 (batch/offline) 强化学习** 兴起的最近几年, Actor-Critic 框架也被重塑以应对“分布偏移”和“价值过估计”问题。保守的 Q 学习 (CQL) 等方法在 Critic 的目标中显式引入保守性正则项, 使得在静态数据集上学习的策略不会高估离线数据中未见动作的价值, 这类思想有效地将 Actor-Critic 的离策略能力扩展到只依赖历史数据、无需在线交互的场景——这对现实世界 (机器人、医疗、推荐系统等) 的应用尤为重要。隐式 Q 学习 (IQL)、AWAC 等方法也在这一方向上提出了不同的策略改进与估计技巧。

另一条与 Actor-Critic 密切相关的发展是**模型-基 (model-based) 与混合方法**的再兴。以 MBPO 为代表的工作表明, 把短期的模型生成轨迹与离策略 Actor-Critic (例如 SAC) 结合, 可以在不显著引入偏差的情况下提高样本效率: 模型负责生成额外的、短 horizon 的训练数据, Actor 与 Critic 在真实与合成数据的混合缓冲区上训练, 从而显著减少对真实交互的需求。该方向把模型学习与 Actor-Critic 的稳定优化结合, 为样本匮乏或真实交互昂贵的应用提供可行路径。

在 **多智能体与复杂交互** 的背景下, Actor-Critic 的架构被自然扩展为协作/对抗场景下的定制化设计。MADDPG 等工作提出了 centralised-training with decentralised-execution 的范式: Critic 在训练时可观察到全局信息以缓解非平稳性, 但 Actor 仍然在执行时仅依赖局部观测, 从而在多智能体系统中取得了良好效果。多智能体领域还催生了对稳定学习、信用分配与规模化训练的诸多新问题与方法, 这对 Actor-Critic 的设计提出了新需求。

Actor-Critic 的研究与应用呈现出几条明显趋势。第一是**稳健性与可靠性**成为核心议题: 在真实世界部署时, 函数逼近误差、分布偏移、稀疏奖励与安全约束都要求算法在保证性能的同时也要可解释、可校验。第二, **离线与自监督学习的融合**将持续推进: 如何在大规模静态数据上安全地学习高性能策略, 并结合自监督表示学习来提升样本效率, 是当前活跃的研究方向。第三, **大规模分布式训练与工程实践**会继续演进, 硬件与系统层面的优化 (如集中式推理、通信压缩、可重复性控制) 正在把研究算法转化为可重复、低成本的训练流水线。第四, 跨学科方向 (例如将因果推断、控制理论或安全约束嵌入 Actor-Critic) 正在逐步形成若干更强的理论保障与实际可用性路径。

8. 强化学习总结

强化学习 (Reinforcement Learning, RL) 旨在通过与环境的交互学习最优决策策略, 是机器学习中连接“感知—决策—行动”的关键环节。从历史脉络看, 强化学习的发展大致经历了三个阶段: 以表格方法为主的**早期强化学习**, 以函数逼近为核心的**深度强化学习**, 以及近年兴起的大模型与人类反馈结合的**强化学习**。其研究目标从最初的理论建模逐步扩展到复杂环境中的自主智能体学习, 成为智能决策研究的重要基础。

1950s-1980s	——► 理论奠基阶段
	Bellman 动态规划 (1957)
	Sutton & Barto 提出时序差分学习 (1988)
1980s-2000s	——► 值函数与策略方法确立
	Q-Learning (Watkins, 1992)
	REINFORCE (Williams, 1992)
	Actor-Critic 框架 (Konda & Tsitsiklis, 2000)
2013-2016	——► 深度强化学习兴起
	DQN (Mnih et al., 2015) 实现从像素到动作的端到端学习
	A3C、DDPG 等并行与连续控制算法出现
2017-2020	——► 高效与稳定性提升

- | PPO (Schulman et al., 2017) 简化策略优化
- | Rainbow、SAC 等集成与熵正则化方法出现
- | MuZero、Dreamer 开启模型化学习浪潮

|

2020s-至今 ——► 大模型与强化学习融合

- | Offline RL (CQL, IQL) 适用于离线数据场景
- | Decision Transformer 实现序列建模统一框架
- | RLHF (InstructGPT) 推动人类反馈驱动的智能决策

|

未来展望 ——► 世界模型、可解释性、安全性与通用智能体

从方法体系上看，强化学习的主要思路可分为三大类。**基于价值的方法**（如 SARSA、Q-Learning）通过估计状态或状态—动作价值函数来指导策略更新，在离散状态与动作空间下具有良好的收敛性质与直观性；**基于策略的方法**（如 REINFORCE、PPO）则直接对参数化策略进行优化，适合连续动作空间与高维控制任务，但易受梯度方差与训练不稳定问题影响；**Actor-Critic 方法**将二者结合，由 Critic 评估价值、Actor 更新策略，在稳定性与效率之间取得平衡。随着深度学习的引入，DQN（Deep Q-Network）将卷积神经网络用于价值函数近似，实现了高维感知输入下的端到端学习，标志着强化学习进入深度时代。

进入近几年，强化学习的研究重点逐渐转向**稳定性、效率与泛化能力**。在策略优化方向，Proximal Policy Optimization (PPO) 通过限制新旧策略分布的变化幅度提高了训练稳定性，而 Soft Actor-Critic (SAC) 在最大熵框架下兼顾回报与探索，显著提升了连续控制任务的表现。在值函数学习方向，Rainbow 将多种改进（优先经验回放、双 Q、分布式 Q 等）整合一体，形成性能强劲的深度值学习体系。与此同时，MuZero、Dreamer 等模型化强化学习 (Model-based RL) 方法通过学习环境的潜在动力学模型或在潜空间中进行“想象”，显著提高了样本效率，推动了从“无模型”向“可规划”的方法转变。

在实际应用中，**离线强化学习 (Offline RL)** 成为新的研究热点。该方向针对无法与环境交互、只能利用历史数据的情形（如医疗决策、推荐系统、机器人控制等），通过保守估计与分布约束防止策略在分布外动作上过度乐观。典型方法如 Conservative Q-Learning (CQL) 与 Implicit Q-Learning (IQL)，为强化学习在安全敏感场景的落地提供了理论与实践基础。同时，基于序列建模的 Decision Transformer 将强化学习任务转化为序列预测问题，借助 Transformer 架构实现了从大规模序列建模到决策学习的统一，为深度强化学习与预训练模型的融合开辟了新方向。

值得注意的是，强化学习已在**人工智能对齐与人类反馈学习 (RLHF)** 领域发挥关键作用。通过引入人类偏好或排序数据训练奖励模型，再以强化学习方法微调策略模型，使系统行为更符合人类意图。这一框架最初由 Christiano 等人提出，并在 InstructGPT 等大规模语言模型中得到广泛应用，标志着强化学习方法从控制与博弈扩展到自然语言理解与生成领域，为“可控人工智能”提供了重要路径。

展望未来，强化学习的发展将呈现多元化趋势。一方面，**样本效率与稳定性**仍是核心挑战，模型化方法、世界模型 (World Models) 与模拟—现实迁移 (Sim-to-Real) 研究将持续深化；另一方面，**离线学习、安全与保守策略优化**将成为实际部署的关键技术。同时，随着大规模序列模型与强化学习的融合加深，“**大模型 + 强化学习**”的框架有望统一感知、推理与决策，实现从语言到行动的智能体体系。最终目标是构建能够在复杂、动态、非平稳环境中安全学习、自主决策并具备泛化能力的智能系统。

强化学习方法

- 基于价值 (Value-based)
 - | —— 表格方法: SARSA, Q-Learning
 - | —— 深度方法: DQN, Double DQN
- |
- 基于策略 (Policy-based)

```

|   |—— 随机策略: REINFORCE, TRPO, PPO
|   |—— 确定性策略: DPG, DDPG
|
|—— Actor-Critic 类
|   |—— A2C / A3C
|   |—— DDPG
|   |—— SAC (最大熵Actor-Critic)
|
|—— 模型化强化学习 (Model-based RL)
|   |—— Dyna-Q
|   |—— Dreamer
|   |—— MuZero
|
|—— 离线强化学习 (Offline RL)
|   |—— CQL
|   |—— IQL
|
|—— 大模型与反馈强化学习
|   |—— Decision Transformer
|   |—— RLHF (人类反馈强化学习)

```

参考文献

- **强化学习经典教材**: Sutton R S, Barto A G. Reinforcement Learning: An Introduction. Cambridge: MIT Press, 1998.
- **SARSA**: Rummery GA, Niranjan M. On-line Q-learning using connectionist systems. (1994) Cambridge Univ. Engineering Dept. Technical Report.
- **REINFORCE**: Williams RJ. Simple statistical gradient-following algorithms for connectionist reinforcement learning. Machine Learning, 1992, 8(3-4): 229–256.
- **Actor-Critic**: Konda V R, Tsitsiklis J N. Actor-Critic Algorithms. NIPS, 2000.
- **DQN**: Mnih V, Kavukcuoglu K, Silver D, et al. Human-level control through deep reinforcement learning. Nature, 2015, 518(7540): 529–533.
- **DDPG**: Lillicrap T P, Hunt J J, Pritzel A, et al. Continuous control with deep reinforcement learning. arXiv:1509.02971, 2015.
- **A3C**: Mnih V, Badia A P, Mirza M, et al. Asynchronous methods for deep reinforcement learning. arXiv:1602.01783, 2016.
- **PPO**: Schulman J, Wolski F, Dhariwal P, et al. Proximal Policy Optimization Algorithms. arXiv:1707.06347, 2017.
- **Rainbow**: Hessel M, Modayil J, Van Hasselt H, et al. Rainbow: Combining improvements in deep reinforcement learning. arXiv:1710.02298, 2017. arXiv
- **SAC**: Haarnoja T, Zhou A, Abbeel P, Levine S. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning. arXiv:1801.01290, 2018.

- **AlphaGo Zero**: Schrittwieser J, Antonoglou I, Hubert T, et al. Mastering Atari, Go, chess and shogi by planning with a learned model. Nature, 2020, 588: 604–609.
- **在潜在空间中进行想象以提高样本效率**: Hafner D, Lillicrap T, Fischer I, et al. Dream to Control: Learning Behaviors by Latent Imagination. arXiv:1912.01603, 2019.
- **Transformer RL**: Chen L, Tworek J, Jun H, et al. Decision Transformer: Reinforcement Learning via Sequence Modeling. arXiv:2106.01345, 2021.
- **离线 RL**: Kumar A, Zhou A, Tucker G, Levine S. Conservative Q-Learning for Offline Reinforcement Learning. NeurIPS, 2020.
- **离线 RL**: Kostrikov I, Wu Y, Nachum O. Offline Reinforcement Learning with Implicit Q-Learning. arXiv:2110.06169, 2021.
- **RLHF 基础**: Christiano P F, Leike J, Brown T, et al. Deep reinforcement learning from human preferences. arXiv:1706.03741, 2017.
- **InstructGPT / RLHF**: Ouyang L, Wu J, Jiang X, et al. Training language models to follow instructions with human feedback. arXiv:2203.02155, 2022.